

UNIVERSITY OF THESSALY
SCHOOL OF ENGINEERING
DEPARTMENT OF ELECTRICAL AND COMPUTER ENGINEERING

Investigation of Machine Vibration Analysis System Design

Diploma Thesis

Vasileios Dimitriou

Supervisor: Karakonstantis Georgios

Month 2022



UNIVERSITY OF THESSALY
SCHOOL OF ENGINEERING
DEPARTMENT OF ELECTRICAL AND COMPUTER ENGINEERING

Investigation of Machine Vibration Analysis System Design

Diploma Thesis

Vasileios Dimitriou

Supervisor: Karakonstantis Georgios

Month 2022



ΠΑΝΕΠΙΣΤΗΜΙΟ ΘΕΣΣΑΛΙΑΣ

ΠΟΛΥΤΕΧΝΙΚΗ ΣΧΟΛΗ

ΤΜΗΜΑ ΗΛΕΚΤΡΟΛΟΓΩΝ ΜΗΧΑΝΙΚΩΝ ΚΑΙ ΜΗΧΑΝΙΚΩΝ ΥΠΟΛΟΓΙΣΤΩΝ

**Διερεύνηση Σχεδίασης Συστημάτων Ανάλυσης Δονήσεων
Μηχανών**

Διπλωματική Εργασία

Βασίλειος Δημητρίου

Επιβλέπων/πουσα: Καρακωνσταντής Γεώργιος

Μήνας 2022

Approved by the Examination Committee:

Supervisor **Karakonstantis Georgios**

Rank/position of Supervisor, Department of Electrical and Computer
Engineering, University of Thessaly

Member **Name Surname of Member 1**

Rank/position of Member 1, Department/Institution of Member 1

Member **Name Surname of Member 2**

Rank/position of Member 2, Department/Institution of Member 2

DISCLAIMER ON ACADEMIC ETHICS AND INTELLECTUAL PROPERTY RIGHTS

«Being fully aware of the implications of copyright laws, I expressly state that this diploma thesis, as well as the electronic files and source codes developed or modified in the course of this thesis, are solely the product of my personal work and do not infringe any rights of intellectual property, personality and personal data of third parties, do not contain work / contributions of third parties for which the permission of the authors / beneficiaries is required and are not a product of partial or complete plagiarism, while the sources used are limited to the bibliographic references only and meet the rules of scientific citing. The points where I have used ideas, text, files and / or sources of other authors are clearly mentioned in the text with the appropriate citation and the relevant complete reference is included in the bibliographic references section. I also declare that the results of the work have not been used to obtain another degree. I fully, individually and personally undertake all legal and administrative consequences that may arise in the event that it is proven, in the course of time, that this thesis or part of it does not belong to me because it is a product of plagiarism».

The declarant

Vasileios Dimitriou

Diploma Thesis

Investigation of Machine Vibration Analysis System Design

Vasileios Dimitriou

Abstract

The abstract includes the scientific area and the purpose - subject of the thesis, the methodology, the main steps followed and the main results obtained. The total extent of the abstract will be up to one page. finding and conclusions to be highlighted.

A thesis written in english should include an english and a greek abstract.

Keywords:

term, term, . . . , term

Διπλωματική Εργασία

Διερεύνηση Σχεδίασης Συστημάτων Ανάλυσης Δονήσεων Μηχανών

Βασίλειος Δημητρίου

Abstract

Η περίληψη περιλαμβάνει την επιστημονική περιοχή και το σκοπό - αντικείμενο της εργασίας, τη μεθοδολογία, τα κύρια βήματα που ακολουθήθηκαν και τέλος τα κύρια αποτελέσματα. Η συνολική έκταση της περίληψης θα είναι μέχρι μία σελίδα.

Διπλωματική εργασία γραμμένη στα ελληνικά, θα πρέπει να περιλαμβάνει ελληνική και αγγλική περίληψη.

Λέξεις-κλειδιά:

όρος, όρος, . . . , όρος

Contents

Abstract	x
Abstract	xi
Contents	xiii
List of Figures	xvii
Abbreviations	xix
1 Introduction	1
1.1 Object of the Diploma Thesis	1
1.1.1 The Concept of Predictive Maintenance	2
1.1.2 Contribution	2
1.2 Structure of the Thesis	3
2 Background	5
2.1 Introduction	5
2.1.1 The Scope of Predictive Maintenance	5
2.1.2 The Lack of Preventive Maintenance	6
2.1.3 The Reliability	7
2.1.4 Vibration Analysis Instruments	12
2.1.5 Typical Applications of Vibration Analysis	15
2.2 Fundamentals of Bearing Fault Frequencies	16
2.2.1 Unfault and Fault Models	17
2.3 Algorithms and Technologies	19
2.3.1 Algorithm Introduction	19

2.3.2	Signal Processing	20
2.4	Internet of Things	26
2.4.1	Introduction to Internet of Things & Key Technologies	26
2.4.2	Cloud-Oriented Perspective	26
2.4.3	Internet- vs. Things-Oriented Perspectives	27
2.4.4	Semantic-Oriented Approach	28
2.5	Project Implementation: Sensing & Communication	29
2.5.1	The Sensor Specifications	29
2.5.2	The Communication Protocol	30
3	System Engineering and Design	35
3.1	Introduction	35
3.2	Hardware Architecture	36
3.2.1	ADXL335 Accelerometer	36
3.2.2	Arduino Uno	38
3.2.3	Raspberry Pi 4	42
3.2.4	The whole set-up	45
3.2.5	Network Infrastructure Specifications	46
3.3	System Requirements	47
3.3.1	Functional Requirements	47
3.3.2	Non-Functional Requirements	48
3.3.3	Network Requirements	48
4	Implementation	51
4.1	Implementation Introduction	51
4.2	Software Architecture and Data Flow	52
4.2.1	The Arduino UNO Algorithm	52
4.2.2	The Raspberry Pi 4 Algorithm	55
4.2.3	Vercel Implementation and Supabase Integration	60
5	Results	65
5.1	Introduction	65
5.2	System Verification and Validation	66
5.2.1	Arduino to Raspberry Pi Data Transmission	66

5.2.2	Raspberry Pi to Supabase and System Performance	67
5.3	Data Depiction and Visualization	69
6	Conclusions	73
6.1	Summary and Conclusions	74
6.2	Future Research	75
	Bibliography	77
	APPENDICES	81
	Appendix	
	Τίτλος Παραρτήματος	83
1	Tables	83
2	Algorithms	88
2.1	Arduino Data Acquisition Algorithm	88
2.2	Raspberry Pi Algorithms	90
2.3	Vercel Webpage Implementation	103

List of Figures

2.1	The Internet of Things paradigm as a result of the convergence of different visions. Source: [1].	7
2.2	Different bearings components (source: [2]).	8
2.3	A bearing's defective outer raceway.	8
2.4	Possible defects of shafts.	9
2.5	Faulty bearings and gears (source: [3]).	9
2.6	Typical vibrometer set-up.	13
2.7	Block diagram of the designed station for vibration analysis (source: [4]). .	14
2.8	ADXL 335 accelerometer.	15
2.9	Fault and unfault spectrum graphs (source: [4]).	18
2.10	Fault tree analysis diagram showing the relationship between machine conditions and specific fault frequencies.	20
2.11	Data collection and assessment algorithm for bearing failure prediction. Source: [5].	21
2.12	Recursive decomposition using even-odd splitting.	26
2.13	The Internet of Things paradigm as a result of the convergence of different visions. Source: [6].	28
2.14	The UART block diagram. Source: [7].	31
3.1	The concept layout.	36
3.2	Arduino schematic pinout (refer to Tables .1 and .2 for details).	41
3.3	40-PIN GPIO of Raspberry Pi 4 (source [8]).	44
3.4	Hardware architecture with signal flow.	47
4.1	End-to-end data flow with protocols	52

5.1	Transmitting system operational validation.	68
5.2	Visualization of the X-axis vibration data. The initial view of the Vercel dashboard shows the dynamically loaded measurement data from Supabase, where the healthy baseline (blue) is compared against the current state (green).	70
5.3	Visualization of the Y-axis vibration data. The chart compares the healthy baseline spectrum (blue reference) with the current sensor measurements (green), enabling the detection of frequency-domain deviations.	71
5.4	Visualization of the Z-axis vibration data. Similar to the X- and Y-axis plots, the frequency response is depicted with both the healthy baseline (blue) and current state (green), highlighting differences between conditions.	71
5.5	Visualization of the overall vibration magnitude. This chart combines all three coordinate axes into a single representation, showing the total vibration energy with the healthy baseline (blue) compared against the current state (green).	71

Abbreviations

βλπ	βλέπε
κ.λπ.	και λοιπά
κ.ο.κ	και ούτω καθεξής
TEI	Τεχνολογικό Εκπαιδευτικό Ίδρυμα
BPF	Band Pass Filter

Chapter 1

Introduction

Motivation and description. Points:

- Importance of maintenance
- Predictive maintenance
- Object of my thesis
- Contribution of my thesis

Εδώ αυτή κάνουμε μια γενική περιγραφή του χώρου εφαρμογής της διπλωματικής. Αναφέρουμε τα χαρακτηριστικά του χώρου και καταλήγουμε στα γενικότερα προβλήματα που αντιμετωπίζει ο χώρος. Η συζήτηση των προβλημάτων θα πρέπει να προϋποθέτει τον αναγνώστη για το τι θα προσπαθήσει να αντιμετωπίσει η διπλωματική, χωρίς ακόμα να αναφερόμαστε συγκεκριμένα στο αντικείμενο της διπλωματικής.

1.1 Object of the Diploma Thesis

Elaboration on a system that predicts the failure of mechanical components based on historical data. This thesis focuses on resolving the issue of unplanned shutdowns due to corrective maintenance of machines that have bearings. Εδώ αναφερόμαστε συγκεκριμένα στο τί θα κάνει η διπλωματική. Αναφέρουμε λεπτομερώς α) τα προβλήματα που θα λύσει (και που ήδη έχουν περιγραφεί γενικά στην προηγούμενη ενότητα), και β) πώς σκοπεύει να τα λύσει. Είναι σημαντικό κάποιος που θα διαβάσει την ενότητα αυτή να καταλάβει σε σημαντικό βαθμό τον σκοπό της διπλωματικής σας και τις τεχνικές δυσκολίες της, χωρίς

να είναι αναγκαίο να δει όλα τα άλλα κεφάλαια. Η ενότητα αυτή θέλει πολύ προσοχή και καλύτερα να τη γράψετε αφού έχετε γράψει όλα τα υπόλοιπα κεφάλαια.

1.1.1 The Concept of Predictive Maintenance

Getting a good grasp on the upcoming required maintenance activities and being aware of what is of utmost importance is vital for the operation of a plant. A grasp on Periodical status checks of different machines is crucial for the maintenance engineers, since this could yield a clear understanding of what priority is at that instance of time. Therefore the prediction of an upcoming machine failure would constitute an invaluable tool.

- Good condition readings
- Periodical readings
- Readings analysis
- online board

1.1.2 Contribution

Εδώ παραθέτουμε αριθμητικά συγκεκριμένες ενέργειες/λύσεις/μεθοδολογίες που παρουσιάζει η διπλωματική και λύνουν τα προβλήματα που υποσχθήκαμε στην προηγούμενη ενότητα ότι θα λύσει η διπλωματική. Συνήθως η υποενότητα αυτή έχει την παρακάτω μορφή:

Η συνεισφορά της διπλωματικής συνοψίζεται ως εξής:

1. Μελετήθηκαν συστήματα κ.λ.π.
2. Υλοποιήθηκαν τρεις αλγόριθμοι υπολογισμού κ.λ.π.
3. Αξιολογήθηκε η επίδοση των αλγορίθμων και βρέθηκε ότι κ.λ.π.
4. Ενσωματώθηκαν οι αλγόριθμοι σε πρότυπο σύστημα κ.λ.π.
5. ...

1.2 Structure of the Thesis

Εδώ περιγράφουμε τα κεφάλαια της διπλωματικής: μία πρόταση για το τί θα έχει κάθε κεφάλαιο. Συνήθως η ενότητα αυτή έχει την παρακάτω μορφή (δεν θα σας πάρει πάνω από μία μεγάλη παράγραφο):

Εργασίες σχετικές με το αντικείμενο της διπλωματικής παρουσιάζονται στο Κεφάλαιο 2. Το Κεφάλαιο ... συζητά θέματα μοντελοποίησης. Στο Κεφάλαιο ... αναπτύσσουμε κ.λ.π.

Για την τελική οργάνωση του κειμένου σας, συμβουλευθείτε τον επιβλέποντα της εργασίας.

Chapter 2

Background

2.1 Introduction

2.1.1 The Scope of Predictive Maintenance

There is a tremendous abundance of applications where the vibration analysis is a prerequisite. Readings and records are of vital importance in the context of the predictive maintenance implementation. Most of the industries nowadays strive to make use of accurate preventive maintenance plans, which requires a well-rounded continuous monitoring system, that keeps data of the whole equipment. The ultimate goal of the preventive maintenance plan is the elimination of the unwanted shut-downs due to a machine failure and the increase of plant reliability.

Most of the machines need bearings because bearings are mechanical components designed to support rotating shafts and minimize friction. The bearings failure may have a wide variety of causes, lubrication shortage, seal failure, high temperatures, misalignment are some of the causes among others. Therefore vibration analysis focuses on predicting all these possible causes of bearings failure. Some of the preventive actions against the bearings failure are described bellow.

Proper lubrication plays a vital role in bearing longevity. Rather than choosing oils based solely on moderate operating temperatures, engineers must consider factors like bearing speed, size, and load. Maintaining an adequate oil film is critical, and minimum viscosities are established based on bearing type. This oil film ensure that no metal-to-metal contact takes place for a predetermined load and speed. If these thresholds aren't met, metal-to-metal

contact occurs, significantly reducing reliability [1].

Alignment is another critical cause of bearings failure. A misalignment of some equipment -typically consisted of a motor and the main equipment- leads to bearings failure. For this purpose vibration and temperature monitoring are significant. For this reason, alignment practice upon machine installation is mandatory. According to Bloch the proper initial shaft alignment is necessary in any case and it is proven that the most advantageous technique to do so is the laser optic one [1].

A detailed alignment procedure is also presented by Bloch. On Figure 2.1 is depicted a typical aligning tool set-up. The laser headers must be installed on machined surfaces, which entails that it will be settled on the motor shaft and the shaft of the rotated mechanism. This laser mechanism has an emitter system, that produces a laser beam, and the laser beams are received by some receivers. Usually a three reference point alignment is what the optic laser machines offer, and this procedure takes place while the motor and the rotated machine are coupled with each other.

The laser mechanism set-up (laser emitters and receivers) are connected to some alignment computer, which is powered by some special software. This yields some reading and some alignment correction recommendations. The alignment criteria depend on the mechanical design of the equipment and the rotating speed as well [1].

However there could be some more causes of machinery failure, specifically a loose machinery base or tightening bolt is another that could lead to a components failure. Hence, the way to alleviate this issue is to align and tighten the machinery back to the frame.

How does vibration diagnosis contribute to operational reliability and cost efficiency in engineering applications? The answer to this question is that the implementation of vibration-analysis-based predictive maintenance helps in the direction of troubleshooting the cause of some issue. In addition, raising awareness on some upcoming failure leads to proper planning, which can prevent greater equipment damage.

2.1.2 The Lack of Preventive Maintenance

It is worth mentioning that there is a large number of possible faults that may lead to vibration, when the preventive maintenance is not implemented properly. The most common defect that is encountered on machinery is this of the bearing failure (see Figure 2.2 some bearings components and on Figure 2.3 a bearing's component defect).

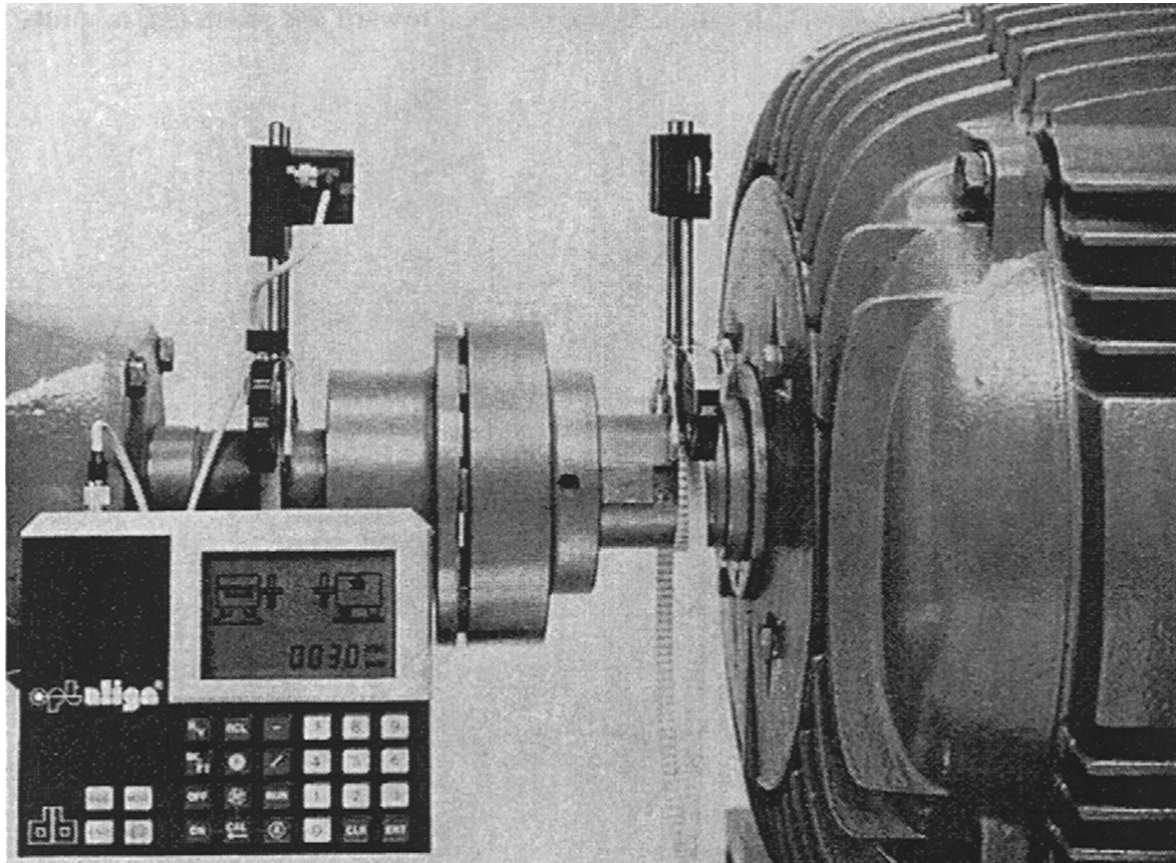


Figure 2.1: The Internet of Things paradigm as a result of the convergence of different visions. Source: [1].

A bearing's defective component will certainly lead to greater problems. Specifically, this could lead to a severe shaft wearing or even thread damage (see 2.4). In some other cases this could lead to a bearing housing damage.

Chen et al included in their research some more photos related to faulty bearings and gearsets [3].

2.1.3 The Reliability

To begin with, the ultimate objective of the preventive maintenance concept is to improve the dependability. In essence, it is a concept that encompasses different subconcepts like Reliability, Availability, Maintainability and others. The reliability interprets the successful uninterrupted operation of an adequately-performing device or an engineering system. This entails that the operating system must retain not only a low Failure rate, but the most important trait of a device in operation must be the absence of failures so as to claim the title of a "reliable system".

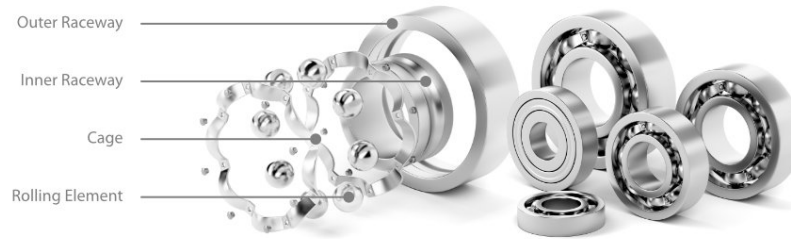


Figure 2.2: Different bearings components (source: [2]).



Figure 2.3: A bearing's defective outer raceway.

According to Martz, "reliability analysis is the engineering and statistical methodology used to assess the unknown reliability of a device. It may be used qualitatively as a means for identifying the various failure modes and causes, or it may be applied quantitatively, using appropriate failure data, to provide a numerical estimate of the unknown reliability" [9]. Hence, it is worth it utilizing the aforementioned metrics (MTTF and MTBF), when contemplating reliability, since these constitute the actual metrics of reliability.

The initial aspect of the theoretical background to be elaborated upon is Mean Time to Failure (MTTF) is a fundamental reliability metric used to estimate the average operational lifetime of non-repairable components and systems. In the context of this very article, the main focus of the study is mechanical components (e.g bearings, gears) as well as a system-machine in operation.

Some of the most important functions that are being used in the context of the metrics interpretation are the following:

- $f(x)$ is the probability density function of the time to failure.



Figure 2.4: Possible defects of shafts.



Figure 2.5: Faulty bearings and gears (source: [3]).

- $R(t)$ represents the probability that the device is still operational at time t [9].
- $F(t)$ is the cumulative distribution function of the time to failure, representing the probability that the system has failed by time t .

Interpretation of the MTTF

In this regard the MTTF can be modeled as the integral of the reliability function $R(t)$ over time, from zero to infinity:

$$\text{MTTF} = \int_0^{\infty} R(t) dt$$

Where:

$$R(t) = \Pr(T > t) = \int_t^{\infty} f(x) dx = 1 - F(t)$$

The MTTF interprets the expected time until failure for a large number of identical devices under similar operating conditions.

MTTF is used to predict the average lifespan of a device, providing critical information for maintenance planning, warranty determination, and lifecycle management. A higher MTTF indicates a more reliable device with a longer expected operational life. This metric is crucial for designing reliable products and for making informed decisions about product development and quality assurance.

Interpretation of the MTBF

The MTBF is another critical metric used in reliability engineering, particularly for repairable systems. It represents the average time between failures for a system or component. For repairable systems, MTBF is calculated as the sum of the operational time divided by the number of failures:

$$\text{MTBF} = \frac{\text{Total operational time}}{\text{Number of failures}}$$

In this case, the reliability function would be:

$$R(t) = e^{-\lambda t}$$

Mathematically, MTBF is the inverse of the failure rate λ for systems with a constant failure rate [9]:

$$\text{MTBF} = \int_0^{\infty} t f(t) dt = \int_0^{\infty} R(t) dt = \frac{1}{\lambda}$$

MTBF helps in understanding the reliability of a system over time and is essential for planning maintenance schedules, improving system design, and ensuring continuous operation with minimal downtime. A higher MTBF indicates a more reliable system that experiences fewer failures over its operational life.

Interpretation of the Failure Rate

The Failure rate, often denoted as λ , is defined as the frequency with which an engineered system or component fails. This metric is typically expressed in terms of the number of failures per unit time.

In mathematical term the Failure rate can be expressed as follows:

$$\lambda(t) = \frac{f(t)}{R(t)} = \frac{f(t)}{1 - F(t)}$$

There is three failure rate cases that may be encountered:

- **Constant Failure Rate:** A constant λ suggests that the system is in its "useful life" period, where the rate of failure is steady and does not change over time.
- **Increasing Failure Rate (IFR):** An increasing $\lambda(t)$ indicates that the system is aging and becoming more prone to failure.
- **Decreasing Failure Rate (DFR):** A decreasing $\lambda(t)$ is often observed during the initial "infant mortality" period, where early failures occur, but the remaining items are more reliable over time.

Constant Failure Rate

For systems with an exponential distribution of failure times (i.e., a constant failure rate), the failure rate λ is constant over time:

$$\lambda = \text{constant}$$

In this case, the reliability function $R(t)$ and the probability density function $f(t)$ are given by:

$$R(t) = e^{-\lambda t}$$

$$f(t) = \lambda e^{-\lambda t}$$

Interpretation of the Availability

Availability (A) is typically expressed as a percentage and is calculated using the following formula[10]:

$$A = \frac{MTBF}{MTBF + MTTR} \times 100\%$$

Where:

- MTBF (Mean Time Between Failures): The average time interval between two consecutive expected failures of a system component or mechanism.
- MTTF (Mean Time To Repair): The average time required to repair a failed system component or mechanism and restore it to operational status.

2.1.4 Vibration Analysis Instruments

There is a great number of vibration monitoring products. Some of them are listed here-under:

- amplitude velocity and acceleration (range of motion) vibrometers
- frequency (rate of oscillation repetition) vibrometers
- accelerometers

The accelerometers and vibrometers, both can be remote - cordless sensor units or wired sensor units connected to some computer. The latter device is a smart device powered by some software so as to store and analyze data. In many cases these devices may also offer some online-cloud options, storing data on some database yielding accessibility for multiple users.

The device depicted in the image exemplifies a standard configuration of a vibration meter and an accelerometer (see Figure 2.6).

The accelerometer measures vibration events in mg (or in some cases in mm/s^2 and interprets these events into frequency domain data. While the vibrometer stores reading of vibration velocity (mm/s) of vibration acceleration (mm/s^2).

These devices have the following common traits, they require the machinery size and rotation as an input, so as to proceed with the results extraction. In addition to this, these can execute this calculation on all three directions simultaneously. And to top it all off, they are expensive.

A deeper understanding of the available in the market sensor is given by Lilly et al [11]. They divide vibration equipment into three levels of detail:

- hand-held sensor
- single-channel FFT



Figure 2.6: Typical vibrometer set-up.

- multi-channel FFT

Vibration measurement tools vary significantly in their technical capabilities. Basic handheld devices offer convenient portability for field use but have several limitations. These compact units are confined to single-location measurements and typically operate within a 10-1000 Hz frequency window. Using velocity transducers, they provide only fundamental vibration amplitude data without phase information or sophisticated analytical features.

More advanced single-channel FFT instruments deliver superior performance for spectral examination. These systems cover a broader 0.5-20 kHz frequency span with enhanced resolution, supporting up to 1600 spectral lines. While they enable more thorough vibration spectrum evaluation, their single-input design prevents multi-point correlation studies or phase-based diagnostics.

For comprehensive vibration analysis, multi-channel FFT platforms are necessary. These high-performance systems accommodate multiple measurement points simultaneously across an extensive DC-100 kHz bandwidth. With ultra-fine resolution reaching 6400 spectral lines, they capture precise phase relationships vital for resonance detection and operational deflection pattern visualization. Such systems incorporate specialized functions including rotational speed tracking, coherence verification, and system response characterization - all crucial for detailed mechanical fault investigation and structural dynamic studies.

The primary technical distinctions between these solutions involve: their measurement approach (single versus multiple points), frequency analysis quality (basic versus high-definition), phase measurement availability, and specialized analytical features. Choosing the appropriate tool involves matching the instrument's capabilities to the diagnostic requirements - from preliminary condition checks with hand-held units to in-depth failure analysis using multi-channel systems.

There is also some obsolete set-ups, for instance the station used by Betta et al for their experiment [4]. This station encompasses an acceleration sensor (8710A50M1 by Kistler™) with 5g measuring range, a signal conditioner with adjustable gain (5118B1 by Kistler™) for data translation, a DPC/C40 by Loughborough Sound Images™ for FFT-Analysis and a host PC (see Figure 2.7).

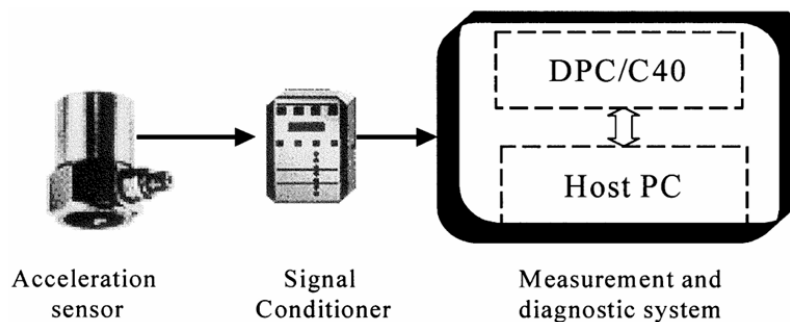


Figure 2.7: Block diagram of the designed station for vibration analysis (source: [4]).

There is some more accessible alternatives. Specifically, there is some small sized low-cost acceleration sensor on breakouts that can be used on related projects (see Figure 2.8).

of noise pollution [11].

The study by Jiao et al. demonstrates the efficacy of time-domain feature extraction combined with Independent Component Analysis (ICA) for gearbox fault diagnosis. Their approach maximizes mutual information to isolate fault-related vibrations from noisy multi-channel measurements, achieving high accuracy in classifying faults such as teeth surface damage and loose foundations [13].

There is also the chance of implementing advanced signal processing techniques, such as envelope analysis and cyclostationary methods, to enhance fault detection in complex systems like helicopters, where high noise and low signal-to-noise ratios are prevalent [14].

In the context of unmanned aerial vehicles (UAVs), Mohd Ghazali and Rahiman employ vibration sensors and artificial intelligence (AI) techniques, including fuzzy logic and neural networks, to detect faults in drone arms. Their framework addresses the challenges of real-time decision-making and integrates IoT for remote monitoring, showcasing the adaptability of vibration analysis to emerging technologies [15]. Meanwhile, Girondin et al. focus on bearing faults in helicopters, proposing redundancy-based harmonic analysis to overcome limitations imposed by low sampling frequencies and environmental noise. Their geometric mean criterion proves robust in identifying fault frequencies despite parasitic vibrations [14].

Theoretical advancements in vibration analysis also emphasize the role of preprocessing steps, such as whitening and Minimum Entropy Deconvolution (MED), to accentuate impulsive features indicative of faults [14]. These methods, combined with machine learning classifiers like Support Vector Machines (SVM) and neural networks, enable automated fault diagnosis with minimal human intervention, as demonstrated in gearbox and bearing applications [13, 15].

Vibration analysis serves as a versatile tool across industries, from traditional rotating machinery to cutting-edge UAVs. Its evolution—driven by innovations in signal processing, AI, and IoT—continues to expand its applicability and precision in fault detection and predictive maintenance.

2.2 Fundamentals of Bearing Fault Frequencies

Li and McKee analyze the failure modes of bearing and yield a brief explanation of what each failure mode entails. It is highlighted by their research that wide-band impulses are

generated when rollers pass over a defect. As far as the diagnostics algorithm is concerned, in this research there is presented statistical parameters that help in finding bearings localized defects. In addition to this, they stress that the frequency-related algorithms like the Hilbert transform are the most prominent in bearing diagnostics.

The most significant challenge that Betta et al had to deal with is that of the faulty cases detection. For this reason, fault and unfault models have been developed, and series of comparisons between the actual model generated by apparatus measurements and these two situational models are being executed. This development constitutes a real challenge since this is the main criterion that identifies the state of the motor [4].

2.2.1 Unfault and Fault Models

Betta et al make use of a model-based approach, which entails that the main concept is comparison-oriented. Specifically, there is the unfault model, which contains a high-amplitude peak at the shaft rotating frequency F_ω . In addition to that, it can be noticed in Figure 2.9 that some other patterns distinct themselves from the previously-mentioned unfault model. Regarding the unbalance model, it is obvious that the amplitude at the first tone is much greater. In the case of the misalignment model, it can be noticed that the tone at $2F_\omega$ has a much greater amplitude while the amplitude at the tone of the shaft speed F_ω remains at a high level. In the case of looseness, there are some more tones prevailing on the spectrum, among the tones related to the shaft speed. To top it all off, the most important vibration spectrum is that of the bearing failure (Refer to Figure 2.10). Every bearing is characterized by four critical frequencies, given by its manufacturer, and these frequencies are associated with the four bearing components (inner and outer ring, balls, and cage) [4]:

- BPFO (Ball Pass Frequency Outer) - outer race failure
- BPFI (Ball Pass Frequency Inner) - inner race failure
- BSF (Ball Spin Frequency) - rolling element failure
- FTF (Fundamental Train Frequency) - cage failure

The procedure to calculate critical frequencies involves two steps: first computing the bearing frequency factors, then deriving the vibration frequencies. The following analysis covers the case where the inner race rotates while the outer race remains stationary [12], [4].

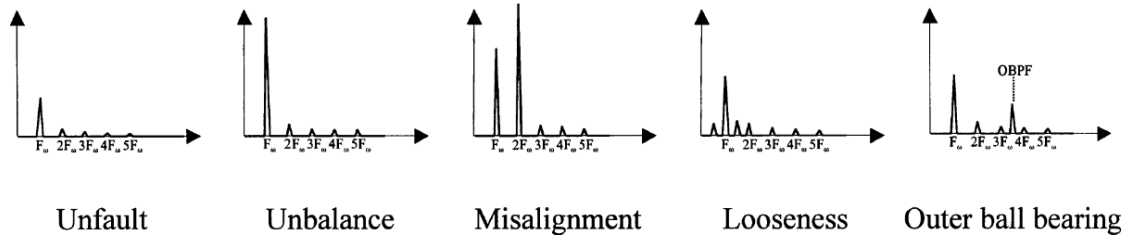


Figure 2.9: Fault and unfault spectrum graphs (source: [4]).

$$F_{OR} = \frac{Z}{2} \left(1 - \frac{d}{D} \cos \alpha \right) \quad (\text{Outer Race Fault}) \quad (2.1)$$

$$F_{IR} = \frac{Z}{2} \left(1 + \frac{d}{D} \cos \alpha \right) \quad (\text{Inner Race Fault}) \quad (2.2)$$

$$F_{FTF} = \frac{1}{2} \left(1 - \frac{d}{D} \cos \alpha \right) \quad (\text{Cage Frequency, Inner Race Rotating}) \quad (2.3)$$

$$F_{BS} = \frac{D}{d} \left[1 - \left(\frac{d}{D} \right)^2 \cos^2 \alpha \right] \quad (\text{Ball Spin Frequency}) \quad (2.4)$$

Then, the aforementioned factors when multiplied with shaft speed (f) give specific critical bearing vibration frequencies:

$$\text{BPFO} = f \times F_{OR} \quad (2.5)$$

$$\text{BPFI} = f \times F_{IR} \quad (2.6)$$

$$\text{BSF} = f \times F_{BS} \quad (2.7)$$

$$\text{FTF} = f \times F_{FTF} \quad (2.8)$$

where:

f	: Shaft rotational speed (Hz)
BPMI	: Ball pass frequency, inner race
BPMO	: Ball pass frequency, outer race
FTF	: Fundamental train frequency
BSF	: Ball spin frequency
Z	: Number of rolling elements
D	: Pitch circle diameter of the bearing (mm)
d	: Rolling element (ball) diameter (mm)
α	: Contact angle ($^{\circ}$)

It is also worth noting that the bearing frequency factors can be obtained through:

- Manufacturer-provided values
- Direct calculation using the above equations (2.1)–(2.4)
- Reverse calculation from measured vibration frequencies (2.5)–(2.8)

To summarize the preceding analysis, Figure 2.10 provides a clear visualization of the fault diagnosis procedure. When analysis reveals elevated frequency tones, a systematic approach must be followed to identify the source of these dominant frequencies. This process essentially involves mapping characteristic vibration patterns to specific machine faults on a frequency spectrum plot.

It must be that a fault-emulation method can be used in such cases to simulate the faulty models. This entails that additional tones can be stressed, and crucial spectra modifications may take place for research and comparison purposes.

2.3 Algorithms and Technologies

2.3.1 Algorithm Introduction

This section aims at yielding a brief introduction into the concept of bearing failure predictive maintenance plan. The set-up utilizes an accelerometer which collects data that are

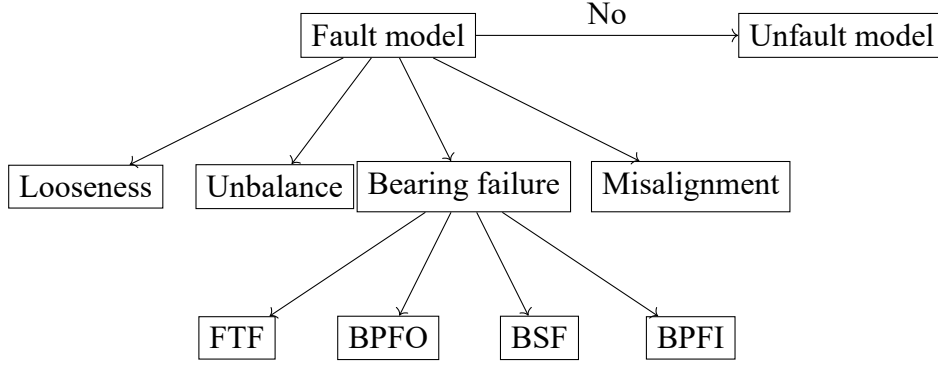


Figure 2.10: Fault tree analysis diagram showing the relationship between machine conditions and specific fault frequencies.

going to be used for model assessment, by the means of Fast-Fourier Analysis.

In most of the actual applications a machine learning implementation is a common practice. A figure that depicts this modeling has been presented by Dharmarathne et al [5].

2.3.2 Signal Processing

Fourier Analysis

In this section a deeper elaboration on Fourier Analysis and Transform is going to take place.

Serafeimidou explains in detail the theoretical background of the Fourier Analysis [16]. For an arbitrary finite range $[-L, L]$ there is a family of functions, which meet the Dirichlet condition, and these functions can be expressed as a trigonometric sequence:

$$f(x) = \frac{a_0}{2} + \sum_{n=1}^{\infty} (a_n \cos(n\pi\omega x) + b_n \sin(n\pi\omega x)), \quad (2.9)$$

where $\omega = \frac{\pi}{L}$.

In addition its constants can be expressed as follows:

$$a_n = \frac{1}{L} \int_{-L}^L f(u) \cos(n\omega u) du \quad (2.10)$$

$$b_n = \frac{1}{L} \int_{-L}^L f(u) \sin(n\omega u) du \quad (2.11)$$

The equation (2.9) needs to be expressed as a valid form within the whole $x \in \mathbb{R}$. For this reason the trigonometric integral (which is called Fourier integral) can be expressed as

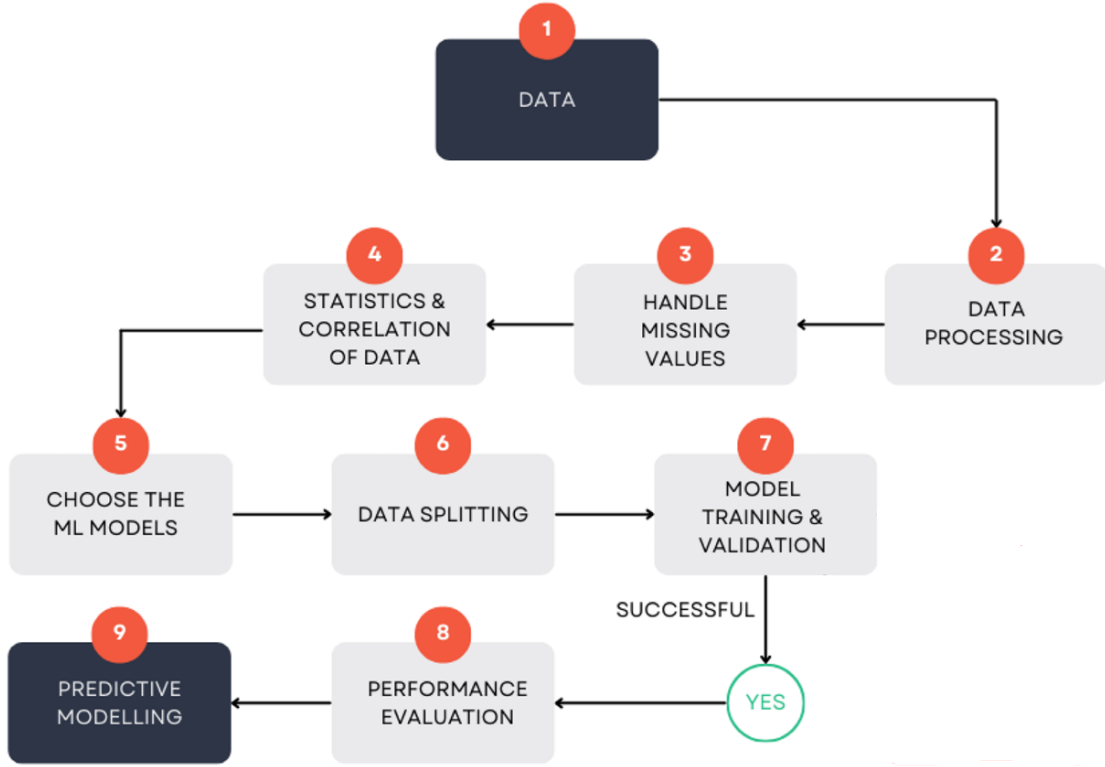


Figure 2.11: Data collection and assessment algorithm for bearing failure prediction. Source: [5].

follows:

$$F(x) = \int_0^{\infty} [A(\omega) \cos(\omega x) + B(\omega) \sin(\omega x)] d\omega \quad (2.12)$$

where:

$$A(\omega) = \frac{1}{\pi} \int_{-\infty}^{\infty} f(u) \cos(\omega u) du \quad (2.13)$$

$$B(\omega) = \frac{1}{\pi} \int_{-\infty}^{\infty} f(u) \sin(\omega u) du \quad (2.14)$$

Assuming that there exist real functions $f(x)$ defined in $\mathbb{R}$ meeting the following points:

- $f(x)$ and its derivative $f'(x)$ are continuous in the range $[-L, L] \subset \mathbb{R}$.
- $f(x)$ is integrable in $\mathbb{R}$:

$$\int_{-\infty}^{\infty} |f(x)| dx < \infty.$$

Hence, given that the aforementioned conditions are met, $A(\omega)$ (see Eq. (2.13)) and $B(\omega)$ (see Eq. (2.14)) tend to attain a real value for ω , which implies that $A(\omega)$ and $B(\omega)$ are defined

for every $\omega \in \mathbb{R}$. In addition to this, the integral of the equation (2.12) tends to attain a real value for x , which ensures that $F(x)$ is defined for every $x \in \mathbb{R}$.

$F(x)$ is identical to $f(x)$ whenever $f(x)$ is a continuous function. There are some special cases when $f(x)$ is not a continuous function, and to deal with these cases, we define:

$$F(x) = \frac{1}{2} \left[\lim_{x \rightarrow x^+} f(x) + \lim_{x \rightarrow x^-} f(x) \right] \quad (2.15)$$

This ensures that $F(x)$ is properly handled even in cases of discontinuity.

All things considered the Fourier Transform $F(\omega) = \mathcal{F}\{F(x)\}$ can be mathematically expressed as follows:

$$F(\omega) = \int_{-\infty}^{\infty} f(t) e^{-i\omega t} dt \quad (2.16)$$

where the frequency can derive from the following form: $\omega = \frac{2\pi jk}{N}$.

In practice, the Discrete Fourier Transform (DFT) boasts an extremely wide variety of applications, facilitating the Fourier Transform when applied to sampled signals.

Hence, the mathematical formulation for determining the amplitude of each frequency, given a waveform $x_0, x_1, \dots, x_{n-1}$ consisting of real values, is structured as follows:

$$X(j) = \sum_{k=0}^{n-1} x(k) e^{-\frac{i2\pi jk}{n}}, \quad \text{for } j = 0, 1, \dots, n-1. \quad (2.17)$$

Fast Fourier Transform (FFT)

The Fast Fourier Transform (FFT) is an efficient algorithm for computing the Discrete Fourier Transform (DFT), expressed by the equation (2.17) and its inverse. One of the most prominent FFT algorithms, the Cooley-Tukey algorithm [17], employs a divide-and-conquer approach to reduce the computational complexity from $\mathcal{O}(n^2)$ (for the naive DFT) to $\mathcal{O}(2n \log n)$, enabling significant speedups for large amount of data.

This reduces complexity from $\mathcal{O}(n^2)$ (naive DFT entails direct implementation of the equation (2.17)) to less than $\mathcal{O}(2n \log n)$ (a Fast Fourier Transform implementation), enabling major time savings for complex and large-scale transforms. It is also worth articulating that the complexity can be interpreted as the number of operations executed [17], [18].

Both algorithms are implemented in major numerical packages including MATLAB and NumPy's routines [19]. What is highlighted by Singleton is that the Cooley-Tukey approach

is considerably faster and decently performing especially when coupled with some additional practices so as to execute the Fast Fourier Transform [18].

The computation time according to the aforementioned article decreases drastically by making use of the Cooley-Tukey algorithm. In addition, this very algorithm performs and adapts better to different applications compared to Good's, Danielson's and Goertzel's approaches.

Cooley and Tukey proved that the sample length can be composite size where it is preferable to make use of sample points sized as power of 2, especially when it comes to personal computers with binary arithmetic system. In addition to this the authors have proven that the power of 3 is formally the most efficient approach, however in the context of the fast Fourier Analysis the so-called Radix-2 implementation is the prominent one [17].

$$N = 2^m \quad (2.18)$$

It is hereby worth it elaboration on the Fast Fourier Transform algorithm [17]. The indices of the equation (2.17) can be expressed as follows:

$$\begin{aligned} j &= j_{m-1} \cdot 2^{m-1} + j_{m-2} \cdot 2^{m-2} + \dots + j_1 \cdot 2 + j_0, \\ k &= k_{m-1} \cdot 2^{m-1} + \dots + k_1 \cdot 2 + k_0, \end{aligned} \quad (2.19)$$

where j_v and k_v are the contents of the respective bit positions within the binary representation, which ensues that they are equal to 0 or 1.

The aforementioned set-up (see eq. (2.17)) leads to the following equation (2.21), encompassing twiddle factors expressed as:

$$W = e^{-2\pi i/N} \quad (2.20)$$

And the Fast Fourier Transform:

$$X(j_{m-1}, \dots, j_0) = \sum_{k_0} \sum_{k_1} \dots \sum_{k_{m-1}} x(k_{m-1}, \dots, k_0) \cdot W^{j \cdot k_{m-1} \cdot 2^{m-1} + \dots + j \cdot k_0} \quad (2.21)$$

where as per the first step:

$$W^{j \cdot k_{m-1} \cdot 2^{m-1}} = W^{j_0 \cdot k_{m-1} \cdot 2^{m-1}} \quad (2.22)$$

Furthermore, in the context of the first calculation, the first array can be expressed as follows:

$$x_1(j_0, k_{m-2}, \dots, k_0) = \sum_{\mathbf{k}_{m-1}} x(\mathbf{k}_{m-1}, \dots, \mathbf{k}_0) \cdot W^{j_0 \cdot \mathbf{k}_{m-1} \cdot 2^{m-1}} \quad (2.23)$$

And hereunder can be contemplated that proceeding to the next innermost-successive sum:

$$W^{j \cdot k_{m-1} \cdot 2^{m-1}} = W^{(j_{l-1} \cdot 2^{l-1} + \dots + j_0) \cdot k_{m-l} \cdot 2^{m-l}} \quad (2.24)$$

followed by the calculation of the arrays:

$$x_l(j_0, \dots, j_{l-1}, k_{m-l-1}, \dots, k_0) = \sum_{\mathbf{k}_{m-1}} x_{l-1}(j_0, \dots, j_{l-2}, \dots, k_{m-l}, \dots, k_0) \cdot W^{(j_{l-1} \cdot 2^{l-1} + \dots + j_0) \cdot k_{m-l} \cdot 2^{m-l}} \quad (2.25)$$

for $l = 1, 2, \dots, m$

And this could constitute a recursive procedure obtaining the successive arrays. And last but not least, this finally could lead to the calculation of X spectrum. It can be contemplated that from the Equation (2.25), there is at all times two consecutive values, the even and the odd value that can be stored at a specific location corresponding to an index of:

$$(j_0 2^{m-1} + \dots + j_{l-1} 2^{m-l} + k_{m-l-1} 2^{m-l-1} + \dots + k_0) \quad (2.26)$$

It is also worth it recalling that m is defined by $N = 2^m$. Since bit positions $2^{(m-l)}$, meaning the j and k coefficients, can only get the values 0 and 1, which ensues that there is multiple-storage locations that can be calculated simultaneously.

The array calculated gives the desired Fourier sums:

$$X(j_{m-1}, \dots, j_0) = A_m((j_0, \dots, j_{m-1})) \quad (2.27)$$

At this point there is a need to simplify the Fast Fourier Transform algorithm. It is mandatory to recall the (2.21) so as to depict the algorithm.

The calculations workload can be reduced, by decomposing the original input sequence $X[k]$, of length N , into two subsequences:

- A sequence consisting of elements at even-numbered indices: $X_{\text{even}}[k]$
- A sequence consisting of elements at odd-numbered indices: $X_{\text{odd}}[k]$

By processing these smaller sequences independently, the algorithm substantially reduces the computational workload. The FFT is applied recursively to each half, and this decomposition continues until each subproblem has length 1, at which point the computation becomes trivial (the amplitude value corresponding to a specific frequency). Furthermore, the results from the smaller FFTs are recombined using twiddle factors (see Equation (2.20)).

An significant property of this algorithm is that that, in the case of $j > \frac{n}{2}$, the values are repeated due to the sinusoidal property of $\exp$, which is known as the symmetry identity. This entails that for the values $j = 0, 1, \dots, n/2 - 1$:

$$X_{\text{even}}(j + n/2) = X_{\text{even}}(j) \quad \text{and} \quad X_{\text{odd}}(j + n/2) = X_{\text{odd}}(j) \quad (2.28)$$

The evens and odds decomposition:

$$X_{\text{even}}(j) = \sum_{k=0}^{n/2-1} x(2k) e^{-\frac{i2\pi jk}{n/2}}, \quad \text{for } j = 0, 1, \dots, n/2 - 1. \quad (2.29)$$

$$X_{\text{odd}}(j) = \sum_{k=0}^{n/2-1} x(2k + 1) e^{-\frac{i2\pi jk}{n/2}}, \quad \text{for } j = 0, 1, \dots, n/2 - 1. \quad (2.30)$$

So from the Equation (2.21):

$$X(j) = X_{\text{even}}(j) + e^{-\frac{i2\pi j}{n}} X_{\text{odd}}(j) \quad (2.31)$$

And this procedure is scalable since each and every X_{even} and X_{odd} can be depleted into a subsequent set of X_{eve} and X_{odd} to the point that the length 1 is reached.

Last but not least, the symmetry identity as described by the Equation (2.28), leads to the following set of equations (see also (2.31)):

$$X(j + n/2) = X_{\text{even}}(j) + e^{-\frac{i2\pi(j+n/2)}{n}} X_{\text{odd}}(j) \quad (2.32)$$

For the values $j = 0, 1, \dots, n/2 - 1$.

The objective of this algorithm is to get to the point that the length is of the sums is 1, meaning that the sums of the Equations (2.29) and (2.30) are applicable for $k=0$. The only remaining exponential terms $e^{-\frac{i2\pi j}{n}}$ in the Equation (2.31) - as previously described by Equation (2.25).

The aforementioned procedure can be further repeated, by breaking the branches problem into sub-branches. The radix-2 DIT algorithm to be formulated as follows on Figure 2.12, depicting the concept of narrowing down the problem.

information across platforms through a unified framework, developing a common operating picture for enabling innovative applications. This is achieved by seamless ubiquitous sensing, data analytics and information representation with Cloud computing as the unifying framework.”

This approach is mostly ”Cloud”-oriented. The authors focus mostly on the concept of ubiquitous information and evolving communication networks. The means to give ground to this very concept is Smart Connectivity. It is fueled by the prevalence of devices enabled by open wireless technology such as Bluetooth, radio frequency identification (RFID), Wi-Fi, and telephonic data services as well as embedded sensor.

Nevertheless, Hardware-sensor breakouts are critically important, as they serve as the primary means of data collection in IoT systems. Middleware and computing tools for data analytics are equally invaluable, as they enable the processing of raw sensor data into actionable insights. Finally, visualization tools and dashboards play a key role in end-user interaction with the IoT environment, while also providing stakeholders with an intuitive and accessible representation of the data [20].

2.4.3 Internet- vs. Things-Oriented Perspectives

As regards the approach of Atzori et al, there is given a different definition by considering two different approaches, the ”Internet”-oriented approach and the ”Things”-oriented approach. The ”Things”-oriented approach targets mainly at Big Data applications, since according to Atzori et al the time, the place and multiple recipients are the most important factors of this approach. In essence the ”Things” that contribute to this perspective are the Radio-Frequency Identification (RFID), Near Field Communications (NFC) and Wireless Sensor and Actuator Networks (WSAN) among others.

The ”Internet”-oriented vision given by Atzori et al, it is highlighted that the IP (Internet Protocol) constitutes the core technology of Internet of Things (IoT). The Internet Protocol (IP) is lightweight, scalable, and capable of connecting smart objects worldwide—even on low-power devices. The authors also introduce Internet Ø, a similar approach that further simplifies IP to enable ”IP over anything,” ensuring seamless IoT integration. The ultimate goal of both strategies is to make all objects addressable, reachable, and interoperable through IP, moving from a basic ”Internet of Devices” to a fully connected ”Internet of Things” [6].

2.4.4 Semantic-Oriented Approach

In addition to the aforementioned perspectives there is an emerging need for representation and storage of the invaluable information, which leads to the "Semantic"-oriented approach (see 2.13). This final approach creates a well-rounded Internet of Things (IoT) concept, yielding more details on it.

The whole concept of the vast majority of different projects requires a semantic approach so as to set some thresholds and yield some feedback or take some action over something. In addition to this the semantic vision according to Atzori et al expresses the need for a representation of the collected data [6].

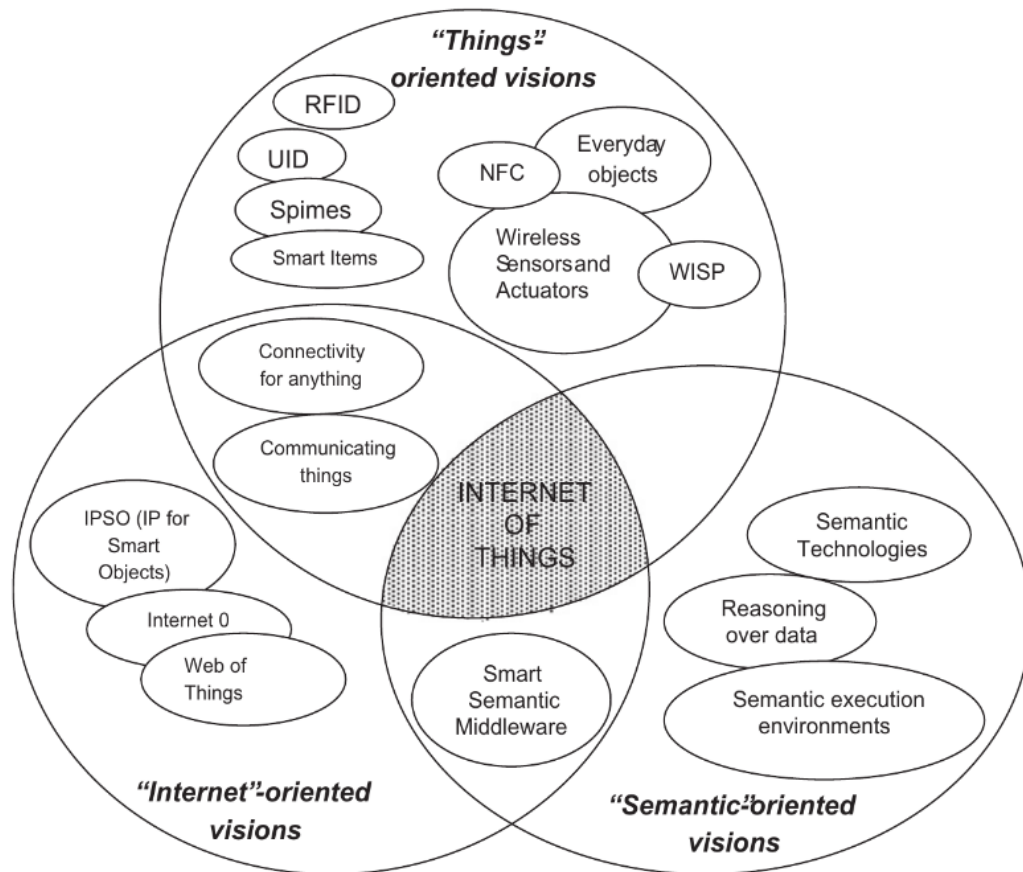


Figure 2.13: The Internet of Things paradigm as a result of the convergence of different visions. Source: [6].

All in all Atzori defines Internet of Things (IoT) as follows:

"Internet of Things constitutes a world-wide network of interconnected objects uniquely addressable, based on standard communication protocols."

2.5 Project Implementation: Sensing & Communication

2.5.1 The Sensor Specifications

As it has been highlighted by the Internet of Things definitions, the sensors are among the most important devices of this technology. The sensing unit in this project utilizes an ADXL335 accelerometer to collect physical acceleration data, which connects to the micro-controller through an analog input interface.

The accelerometer ADXL335 is a single component sensing X, Y and Z, having a wide measuring range in all three axes. Typical applications of this sensor is structure deflection projects. This sensor is the ideal choice when it comes to vibration analysis applications, since it retains a measuring range of $\pm 3g$. This range adequate with respect to vibration analysis because in most of the rotating equipment applications an acceleration greater than 1,5g indicates a defective component. This entails that the $\pm 3g$ is sufficient in the context of this project. In addition to this, it offers higher resolution, up to 300mV/g for vibration detection, which means it has relatively high sensitivity to small vibrations. As regards its frequency bandwidth, it ranges 0.5-1600Hz. As regards its noise density, it equals to $150 \mu g/\sqrt{Hz}$, which is ideal for low-amplitude vibration waveforms[21].

The Adafruit ADXL335 is a high performance sensor breakout, utilizing capacitors so as to decouple the noise of the power supply and filters om analog outputs for antialiasing and noise reduction.

As for the sensor's theory of operation [21]:

ADXL335 is a polysilicon surface-micromachined structure built on top of a silicon wafer. Polysilicon springs suspend the structure over the surface of the wafer and provide a resistance against acceleration forces Deflection of the structure is measured using a differential capacitor that consists of independent fixed plates and plates attached to the moving mass. The fixed plates are driven by 180° out-of-phase square waves. Acceleration deflects the moving mass and unbalances the differential capacitor resulting in a sensor output whose amplitude is proportional to acceleration. Phase-sensitive demodulation techniques are then used to determine the magnitude and direction of the acceleration.

This combination of mechanical design and signal processing provides robust acceleration measurement without requiring temperature compensation circuits -since this breakout

retains a large tolerance into operation temperatures- or complex calibration procedures [21].

2.5.2 The Communication Protocol

The system architecture of the ADXL335 breakout supports analog communication protocol option, however there is a number of different communication protocols frequently utilized by the Adafruit breakouts:

- **Analog Input:** Currently implemented for direct sensor interfacing (ADXL335)
- **Digital Input:** Available for threshold-based detection applications or PWM
- **SPI (Serial Peripheral Interface):** Suitable for high-speed (up to 10 Mbps) communication with multiple peripherals
- **I²C (Inter-Integrated Circuit):** Enables multi-device communication using only two wires (SDA/SCL)
- **UART (Universal Asynchronous Receiver-Transmitter):** Implemented at a pre-determined baud rate for Arduino-to-Raspberry Pi communication

The Universal Asynchronous Receiver-Transmitter Protocol

The Arduino board is connected to the Raspberry Pi microcomputer via its USB port. The USB port of the Arduino is in essence a UART device, that has two options, not working simultaneously. The first one out of the two is the Rx - Tx pins option and the second one is the USB-cable option. In the case of the Arduino-Raspberry Pi interconnection, the desired set-up requires a USB-cable.

The Universal Asynchronous Receiver-Transmitter (UART) is a peripheral device for asynchronous serial communication. The baud rate and the data format is configurable, based on the project needs. After hands-on try and error, it has been found out that a baud rate of 115200 performs adequately offering a higher speed on data transferring, which is vital for the sake of this very project. The data transferring speed is of vital importance, since the higher the amount of data sent to the database is, the larger the frequency domain data range can be.

The raw UART (transmitted via USB) data is in a binary format, and upon receipt by the Raspberry Pi the data batch needs to be converted into a string format. For this reason it is

mandatory to use UTF-8 (8-bit Unicode Transformation Format) to handle ASCII characters at this point.

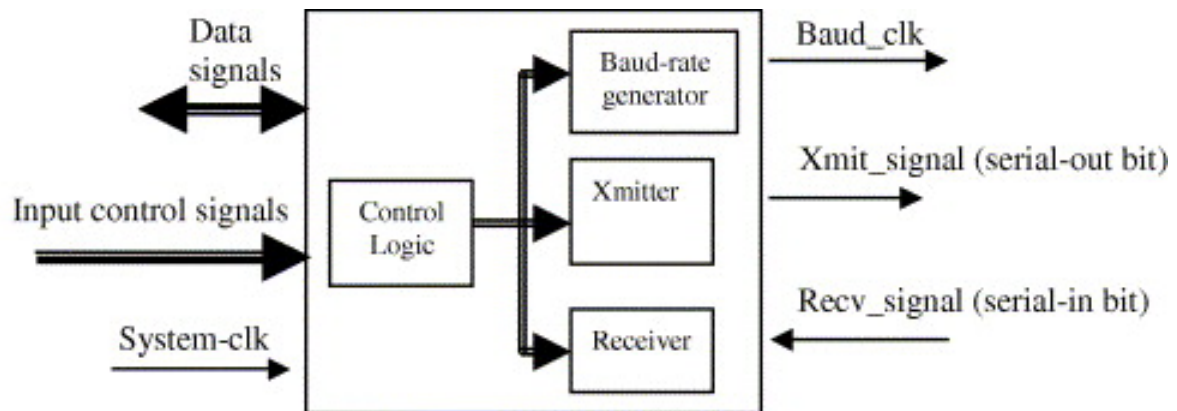


Figure 2.14: The UART block diagram. Source: [7].

A standard UART includes the following key modules (see Figure 2.14):

- **Transmitter** (parallel-to-serial conversion)
- **Receiver** (serial-to-parallel conversion)
- **Baud-rate generator**
- **Control logic**

The baud-rate generator derives a *baud clock* from the system clock. This clock typically operates at 16 times the desired baud rate:

$$\text{baud_clock} = 16 \times \text{baud_rate}$$

This oversampling approach ensures precise timing for bit detection and improves communication reliability.

UART transmits data in structured frames composed of the following elements:

- **Start bit:** Always logic 0
- **Data bits:** Typically 6 to 8 bits (configurable)
- **Optional parity bit:** For basic error detection
- **Stop bits:** Logic 1, usually 1 or 2 bits

The receiver “synchronizes” its internal clock to that of the transmitter’s at the beginning of every data packet received. This allows the receiver to sample the data bit at the bit-cell center. A data packet is composed of 1 start-bit, which is always a logic 0, followed by a programmable number of data bits (typically between 6 and 8). The receiver detects the start bit by sensing a falling edge (logic 1 to 0 transition) and then samples each subsequent bit at the center of its bit period, using the internally generated baud clock. The stop bit must always be logic 1.

UART’s simplicity and robustness make it a widely used solution for embedded systems and serial communication with peripheral devices [7].

Analog Protocol

The Arduino ATmega controllers used for the Arduino contain an onboard 6 channel analog-to-digital (A/D) converter. The converter has 10 bit resolution, returning integers from 0 to 1023. While the main function of the analog pins for most Arduino users is to read analog sensors, the analog pins also have all the functionality of general purpose input/output (GPIO) pins (the same as digital pins 0 - 13) [22].

In this context, it is suggested that the Arduino `analogRead` function work along with some source code that executes calibration function before starts gathering the actual data. This would increase the reliability of the readings with a small toll on the amount of data spent on measuring. The reason why mapping is required is that the Arduino can only return integers from 0 to 1023. Arduino analog pins compare the input voltage on each pin to a reference voltage, which in turn is the operating voltage 5V. So this practice interprets the measurement input from the sensor as a value within the range [0-1023].

$$\text{analogRead}() = \left(\frac{V_{\text{reading}}}{V_{\text{reference}}} \right) \times 1023 \quad (2.33)$$

Another practice that can also increase the reliability of the readings is the usage of some statistical means that would decrease the chances of chances of reading false data. Some exquisite practices are the average value, the median value and the average value after getting rid of the minimum and maximum values.

The aforementioned practices are not mandatory for this context. In addition to this they reduce severely the amount of data within the reading, which leads to smaller amount of data on frequency domain, after carrying out the Fast Fourier Transform. For this reason, raw data

that can go through some mapping-calibration is the ideal practice.

Chapter 3

System Engineering and Design

3.1 Introduction

As it has already been articulated, the objective of this diploma thesis is to study, design, and develop a complete measuring system that integrates a sensor, a microcomputer, and a microcontroller into a coherent and functional unit. The intention behind this effort is not only to demonstrate the feasibility of such a system, but also to provide a framework that can be applied to real-world industrial monitoring scenarios.

The system is specifically conceived for low-cost, real-time vibration monitoring of rotating machinery, a task of significant importance in predictive maintenance and condition monitoring. By capturing the vibrational behavior of machines during operation, valuable insights can be obtained about their mechanical health, potential faults, and overall performance. This contributes directly to reducing downtime, optimizing maintenance schedules, and extending the operational lifespan of equipment.

The chosen architecture relies on a modular set-up that combines hardware and software components of major importance. At its core lies a sensor element capable of detecting vibrations within a predetermined, application-relevant frequency range. The signals acquired are then transferred to a microcontroller, which is tasked with initial data handling, conditioning, and communication. Subsequently, the processed data is relayed to a microcomputer that ensures reliable storage in a structured database, while also providing the computational capacity required for further analysis and processing. In this manner, the system covers the complete cycle of measurement: from data acquisition, to secure storage, and finally to intelligent interpretation.

The following sections of this thesis provide a more detailed overview of the system requirements, addressing both functional requirements (such as measurement accuracy, sampling rates, and communication protocols) and non-functional requirements (such as cost efficiency, scalability, and reliability). Furthermore, a thorough elaboration on the system set-up is presented (see Figure 3.1), offering a comprehensive description of the interaction between hardware modules, data flow, and the software infrastructure that binds them together.

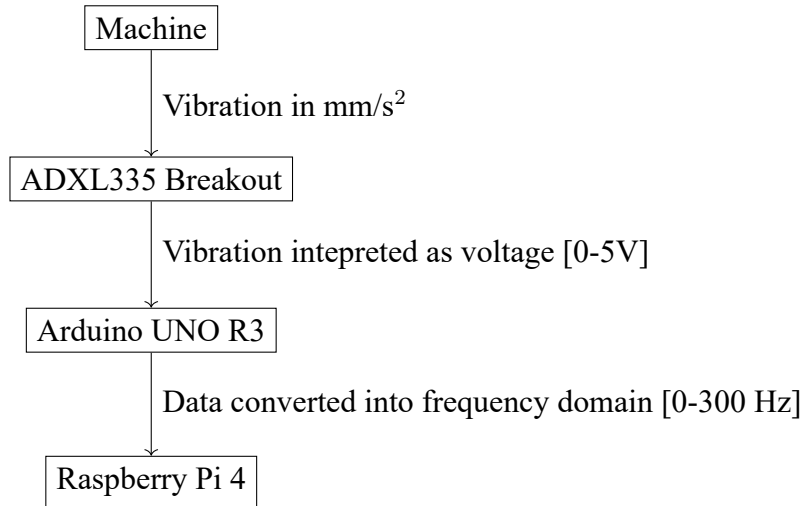


Figure 3.1: The concept layout.

3.2 Hardware Architecture

3.2.1 ADXL335 Accelerometer

The ADXL335 is a small, low-power, 3-axis accelerometer capable of measuring acceleration with a minimum full-scale range of $\pm 3g$. It is designed for motion- and tilt-sensing applications, making it suitable for mobile devices, gaming systems, disk drive protection, image stabilization, and sports and health devices. The device operates on a single supply voltage ranging from $1.8V$ to $3.6V$ and consumes only $350\mu A$ of current, making it ideal for power-sensitive applications [21].

The Adafruit breayout of this very sensor has a power supply at $5V$ which is cut off according to the needs of the sensor. The VCC takes up to $5V$ in and regulates it to $3.3V$ with an output pin. The analog outputs are ratiometric: that means that $0g$ measurement output is always at half of the $3.3V$ output ($1.65V$), $-3g$ is at $0v$ and $3g$ is at $3.3V$ with full scaling in between.

The aforementioned Adafruit ADXL335 accelerometer boasts a wide variety of amendments in comparison with the original stand-alone sensor. Most importantly, this breakout is an ideal choice when it comes to interfacing with 5V microcontrollers like Arduino. It is compatible with microcontrollers because there is pending connections on the breakout. Alternatively, a PCB must be designed so as to solder the small scale and hard to manage pins of the sensor itself.

Another significant feature of the breakouts is that manufacturing research has been conducted and this has lead to the optimal product development. This entails that these breakouts encompass decoupling capacitors, filters and power supply regulation that assures the pinout signals to be as accurate as possible.

It can measure the static acceleration of gravity in tilt-sensing applications, as well as dynamic acceleration resulting from motion, shock, or vibration.

A figure depicting the ADXL335 accelerometer is depicted on Figure 2.8 of Chapter 2. The ADXL335 is housed in a 16-lead LFCSP (Low Profile Chip Scale Package). Table 3.1 describes its key pin functions.

Table 3.1: ADXL335 Pin Functions [21]

Pin Number	Pin Name	Description
2	ST	Self-Test pin. When set to V_S , it generates an electrostatic force to test functionality.
3,5,6,7	COM	Common ground pins.
12	X_{OUT}	X-axis analog output.
10	Y_{OUT}	Y-axis analog output.
8	Z_{OUT}	Z-axis analog output.
14,15	V_S	Supply voltage pins (1.8 V to 3.6 V).
-	Exposed Pad	Not internally connected but should be soldered for mechanical integrity.

Table 3.2 summarizes the key specifications of the ADXL335 accelerometer [21].

All in all, the ADXL335 is widely used in applications requiring precise motion detection and tilt sensing. Its excellent temperature stability and low power consumption make it suitable for portable and battery-operated devices. The device's performance is further enhanced by its low noise density and adjustable bandwidth, allowing for customization based

Table 3.2: ADXL335 Key Specifications [21]

Parameter	Value
Measurement Range	$\pm 3\text{ g}$ (minimum), $\pm 3.6\text{ g}$ (maximum)
Sensitivity	300 mV/g (typical) at $V_S = 3\text{ V}$ (ratiometric)
Zero g Bias	1.5 V (typical) for X and Y axes, 1.5 V (typical) for Z axis
Noise Performance (X,Y axes)	$150\text{ }\mu\text{g}/\sqrt{\text{Hz}}$ (typical)
Noise Performance (Z axis)	$300\text{ }\mu\text{g}/\sqrt{\text{Hz}}$ (typical)
Bandwidth (X,Y)	Adjustable up to 1600 Hz via external capacitors
Bandwidth (Z)	Adjustable up to 550 Hz via external capacitors
Operating Temperature	-40°C to $+85^\circ\text{C}$
Shock Survival	$10,000\text{ g}$ (unpowered or powered)

on specific application requirements [21].

For further details, refer to the ADXL335 datasheet [21].

3.2.2 Arduino Uno

The Arduino UNO represents one of the most accessible and widely adopted micro-controller platforms in the maker community. Built around the reliable ATmega328P architecture, this development board strikes an optimal balance between capability and user-friendliness, serving equally well as an educational tool for novices and a rapid prototyping solution for experienced engineers. Its plug-and-play design eliminates complex setup procedures - users can immediately begin development by connecting via standard USB or utilizing various power input methods including the convenient barrel jack for 9V battery operation.

The specifications of Arduino are listed below [23]:

- **ATmega328P Microcontroller Unit** – Serving as the computational core, this component delivers:
 - Efficient 8-bit AVR RISC processing at 16 MHz clock speed
 - Substantial 32 kilobyte Flash memory for program storage
 - 2 kilobytes of volatile SRAM for dynamic memory operations
 - 1 kilobyte EEPROM for persistent data storage across power cycles

- **System Reliability Features:**

- Automatic Power-On Reset (POR) circuitry for stable initialization
- Brown-Out Detection (BOD) protection against voltage fluctuations

- **Integrated Hardware Peripherals:**

- Dual 8-bit and single 16-bit hardware timers with advanced control capabilities
- Full-duplex USART interface with sophisticated clock generation
- Flexible SPI and I2C serial communication controllers
- Precision analog comparator with configurable reference
- System watchdog for fail-safe operation
- Six-channel PWM generation for analog signal emulation
- Pin-change interrupt system for efficient event handling
- Seamless bridge between USB host and USART peripheral
- Automatic protocol conversion (USB packets ↔ serial frames)
- Built-in drivers for major operating systems

- **USB Interface Controller (ATmega16U2):**

- Dedicated 8-bit processor managing USB protocol
- 16 kilobyte reprogrammable Flash memory
- 512 bytes each of EEPROM and SRAM
- On-chip debugging through debugWIRE interface

- **Input/Output Configuration:**

- Fourteen programmable digital I/O lines (with six supporting PWM)
- Six-channel 10-bit analog-to-digital conversion
- Standard USB Type-B connector for programming and power
- In-circuit serial programming (ICSP) header
- Dedicated hardware reset circuit

- **Power Management System:**

- Wide operating voltage range (2.7-5.5V DC)
- Multiple power input options including regulated 5V USB power

A concise overview of the available pins is given in Table 3.3. For completeness, the full pinout information is provided in the appendix in Tables .1 and .2, which list all analog, digital, and power connections in detail. These complement the schematic representation shown in Figure 3.2.

Table 3.3: Summary of Arduino Pin Functions

Category	Typical Pins	Description
Power Supply	+3.3V, +5V, GND, VIN	Provides operating voltages and ground
Analog Inputs	A0–A5	6 analog channels, some also used for I ² C
Digital I/O	D0–D13	General-purpose digital pins, PWM on some
Communication	SDA, SCL, MOSI, MISO, SCK, SS	Interfaces for I ² C and SPI
Special	Reset, IOREF, AREF	Reset, voltage reference, and analog reference

An exquisite trait of Arduino boards, is that these devices boast a built-in set up of pull-up and pull-down resistors, which is extremely helpful when it comes to implementing projects making use of some sensors.

Moreover, the Arduino UNO cleverly bridges modern USB communication with traditional serial (UART) communication through a two-chip design. The main ATmega328P microcontroller handles the actual programming and serial communication using simple UART protocol, while a secondary ATmega16U2 chip acts as a translator, converting between the computer's USB signals and the microcontroller's UART signals. This translation happens automatically in both directions - when you upload code, the USB chip converts your computer's USB commands into UART signals the main chip understands, and when the Arduino sends serial data back, the USB chip converts it back to USB for your computer. The system creates a virtual COM port on your computer, making the Arduino appear as a simple

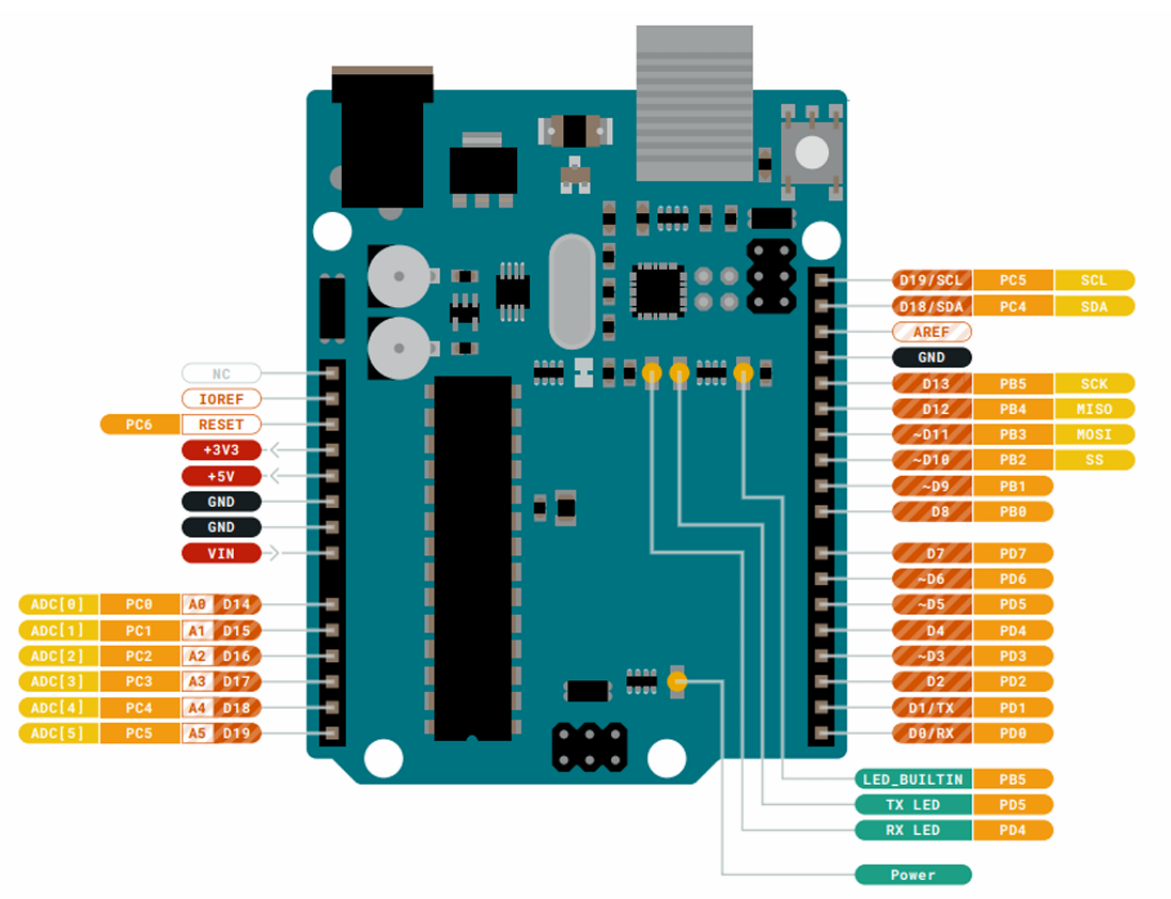


Figure 3.2: Arduino schematic pinout (refer to Tables .1 and .2 for details).

serial device even though it's actually communicating via USB. This elegant solution provides plug-and-play USB connectivity while maintaining compatibility with traditional serial communication, all without requiring any additional components or complex setup from the user. The dual-chip approach also keeps the main processor free from USB protocol handling, allowing it to focus on running your programs efficiently.

The Arduino UNO's serial communication typically operates at standard baud rates ranging from 300 to 115200 bits per second, with 9600 baud being the most commonly used default speed for basic communication. The USB-UART bridge supports higher speeds up to 2 Mbps in theory, but in practice, most applications reliably use 115200 baud or lower. The actual stable maximum baud rate depends on several factors, including the clock accuracy (the UNO's 16MHz crystal provides $\pm 0.3\%$ stability), the quality of USB connections, and the timing requirements of the specific application. For most beginner projects and serial monitoring, 9600 or 115200 baud provide the best balance of speed and reliability, while advanced applications can push higher speeds if properly implemented. The baud rate must match on

both the Arduino and the receiving device for successful communication.

There is also a tremendous amount of source code that is adaptable to different projects. All these sketches can be easily downloaded, stored and modified (or just even tweak some parameters) so as to fit it in each project.

All in all, the UNO is the most popular and well-documented board in the Arduino ecosystem, making it perfect for learning electronics and coding. Its processor ATmega328P chip, in most of the cases is not welded and this allows easy replacement procedure. In addition Arduino UNO is robust, flexible, and beginner-friendly platform for a wide variety of projects. Most importantly it supports easy and robust communication between Raspberry Pi and Arduino.

For further details, refer to the Arduino UNO R3 datasheet [23].

3.2.3 Raspberry Pi 4

The Raspberry Pi is a series of credit-card sized single-board computers developed by the Raspberry Pi Foundation. Designed originally for education, these low-cost computers have evolved into versatile platforms for computing and digital making [8].

There is a plethora of current Raspberry Pi boards (e.g. Raspberry Pi 4 and Raspberry Pi 5). They are all extremely powerful and can cover the needs of highly demanding projects, in terms of memory and computational power [8]. The Raspberry Pi's complete technical specifications are systematically organized in Tables 3.4, 3.5, and 3.6, yielding detailed hardware and software characteristics.

Table 3.4: Key Specifications of Raspberry Pi

Component	Specification
Processor	Broadcom BCM2712 (Pi 5) or BCM2711 (Pi 4) ARM Cortex
Memory	4GB LPDDR4X RAM
Storage	MicroSD card (64GB recommended)
Power Supply	5V USB-C (3A recommended)
Connectivity	Dual-band 802.11ac Wi-Fi, Gigabit Ethernet
Video Output	Dual micro HDMI (up to 4K resolution)
GPIO	40-pin header

Table 3.5: Raspberry Pi OS Features

Feature	Description
OS Type	Debian-based Linux distribution
Desktop	Complete desktop environment
Development	Python tools pre-installed
Hardware Access	GPIO control libraries

Table 3.6: Supported Communication Protocols

Protocol	Description
Digital I/O	General purpose input/output pins
SPI (Serial Peripheral Interface)	High-speed serial communication
I ² C (Inter-Integrated Circuit)	Two-wire serial bus
UART	Asynchronous serial communication

A clear representation of the Raspberry Pi 4 hardware is depicted on the Figure 3.3. Raspberry Pi boasts a wide variety of input and output pins, resembling the capacity of Arduino. It is also programmable, having a source code back-up for multiple hardware options (sensors). The most frequently used coding language for this context is PYTHON.

Table 3.7 details the 40-pin GPIO header configuration, which maintains backward compatibility with previous Raspberry Pi models. The header provides multiple voltage rails (3.3V and 5V) along with dedicated ground pins. The GPIO header serves as the primary interface for hardware projects, supporting everything from simple LED controls to complex sensor networks and custom communication protocols.

The Raspberry Pi 4 Model B offers comprehensive connectivity options as detailed in Table 3.8. The board features dual display outputs through micro-HDMI ports capable of driving 4K resolution simultaneously. For peripheral connectivity, it provides a balanced mix of USB 3.0 and 2.0 ports, with the USB-C power input supporting higher current requirements. The onboard wireless interfaces deliver dual-band Wi-Fi and Bluetooth 5.0 for modern wireless applications.

For further details, refer to the Raspberry Pi 4 Model B datasheet [8].

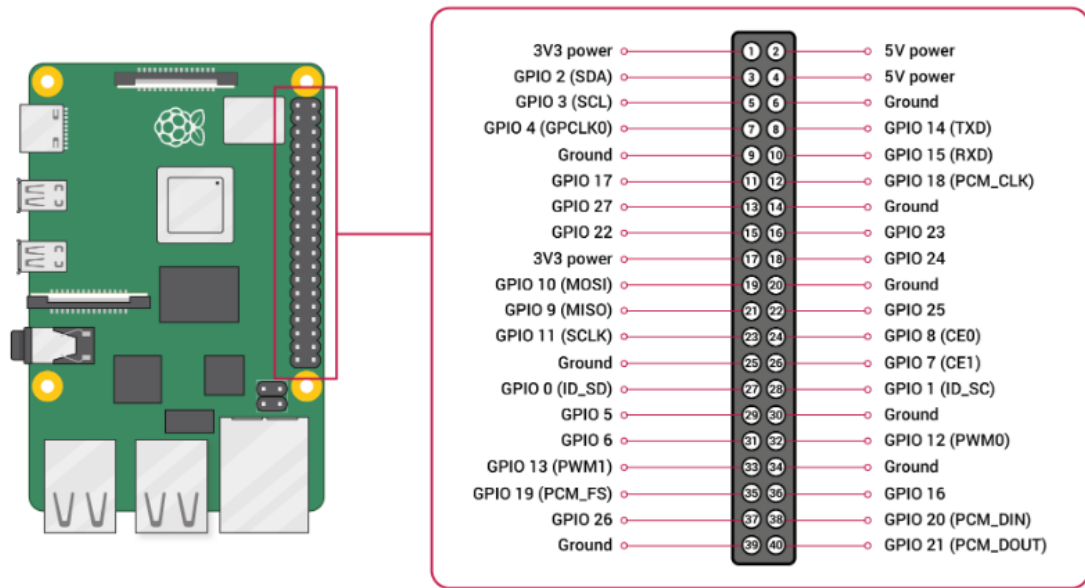


Figure 3.3: 40-PIN GPIO of Raspberry Pi 4 (source [8]).

Table 3.7: GPIO Pinout Details (An extended version of this table is on Appendix - extended version of the following table is the Table .3 (see Chapter 6).

Pin #	Function	Alt Function	Voltage	Notes
1	3.3V	-	3.3V	50mA max
2	5V	-	5V	-
3	GPIO2	SDA1	3.3V	I ² C
4	5V	-	5V	-
5	GPIO3	SCL1	3.3V	I ² C
6	GND	-	-	-
7	GPIO4	-	3.3V	-
...	...	...	...	...
37	GPIO26	-	3.3V	-
38	GPIO20	MOSI1	3.3V	SPI
39	GND	-	-	-
40	GPIO21	MISO1	3.3V	SPI

Table 3.8: Raspberry Pi 4 Model B Ports and Connectors

Port Name	Type	Quantity	Specifications
Primary External Ports			
USB 3.0	Type-A	2	5Gbps, BC1.2 charging support
USB 2.0	Type-A	2	480Mbps
Ethernet	RJ45	1	Gigabit (Realtek RTL8111G)
HDMI	micro-HDMI	2	HDMI 2.0 (up to 4K@60Hz)
Power	USB-C	1	5V/3A (minimum)
Audio	3.5mm jack	1	Stereo output + composite video
GPIO Header			
GPIO	40-pin	1	2×20 pin header, 0.1” spacing
Other Connectors			
CSI	15-pin FPC	1	Camera Serial Interface
DSI	15-pin FPC	1	Display Serial Interface
MicroSD	Slot	1	UHS-I compatible
PoE	4-pin header	1	802.3af support (requires HAT)
Wireless Interfaces			
Wi-Fi	Antenna	1	2.4GHz/5GHz 802.11ac
Bluetooth	Antenna	1	BT 5.0, BLE

3.2.4 The whole set-up

Following the presentation of the different ports of both the Raspberry Pi 4 and the Arduino UNO R3, it is important to also describe the interconnection between these two devices and the way in which data flows through the system. The accelerometer module (Adafruit ADXL335 breakout) generates analog voltage signals corresponding to the acceleration along the three axes (X, Y, and Z). These signals are connected to the analog input pins of the Arduino UNO, which then uses its built-in Analog-to-Digital Converter (ADC) to digitize the incoming data.

After conversion, the Arduino transmits the measurements via serial communication to the Raspberry Pi 4. On the Raspberry Pi side, a Python script is used to set up the serial connection, send requests for new measurements, and receive the acceleration values.

Specifically, the Raspberry Pi sends a command string to the Arduino, which responds with a comma-separated line containing the three acceleration components. The incoming data stream is subsequently retrieved as a text line, converted into a human-readable string, and then parsed into numerical values corresponding to the three acceleration axes. . Finally the data is split into three floating-point numbers representing the X, Y, and Z axes.

The baud rate of the serial communication, which must be pre-set identically on both the Arduino and the Raspberry Pi, plays a crucial role in ensuring reliable data transfer. The baud rate defines the speed at which bits are transmitted per second, and therefore determines how much information can be sent within a given time window. If the chosen baud rate is too low compared to the frequency and volume of data produced by the Arduino, the serial buffer may overflow, leading to delays or data loss. On the other hand, selecting a sufficiently high baud rate allows the system to transmit each line of acceleration data quickly enough to keep up with the sensor output, ensuring that all measurements are captured in real time. For example, at 9600 bps only around 960 bytes per second can be transmitted, which may not be adequate for high-frequency sampling, whereas at 115200 bps more than 11,000 bytes per second can be handled, providing ample capacity for continuous vibration monitoring. In this case, both devices can work well with a baud rate at 115200 bps.

In summary, the flow of information can be described as follows: the ADXL335 breakout produces analog signals sending them to the Arduino UNO samples and digitizes them. Afterwards the data is transmitted through the serial interface to the Raspberry Pi which in turn requests, receives, and parses the acceleration values for further storage and analysis. This interaction ensures a continuous and reliable stream of vibration data, which is schematically represented in Figure 3.4.

3.2.5 Network Infrastructure Specifications

The network infrastructure was configured to ensure reliable and secure data transmission from the sensor system to the cloud database. The router configuration provided optimal performance for IoT data transmission with excellent signal strength and enterprise-grade security protocols, supporting the structured src layout organization of the project.

The data transmission pathway followed a secure flow: *Sensor Device* → *Local Router (192.168.2.1)* → *Internet* → *Supabase API (HTTPS)*. All communications used HTTPS with TLS 1.3 encryption and API key authentication, consistent with the security practices im-

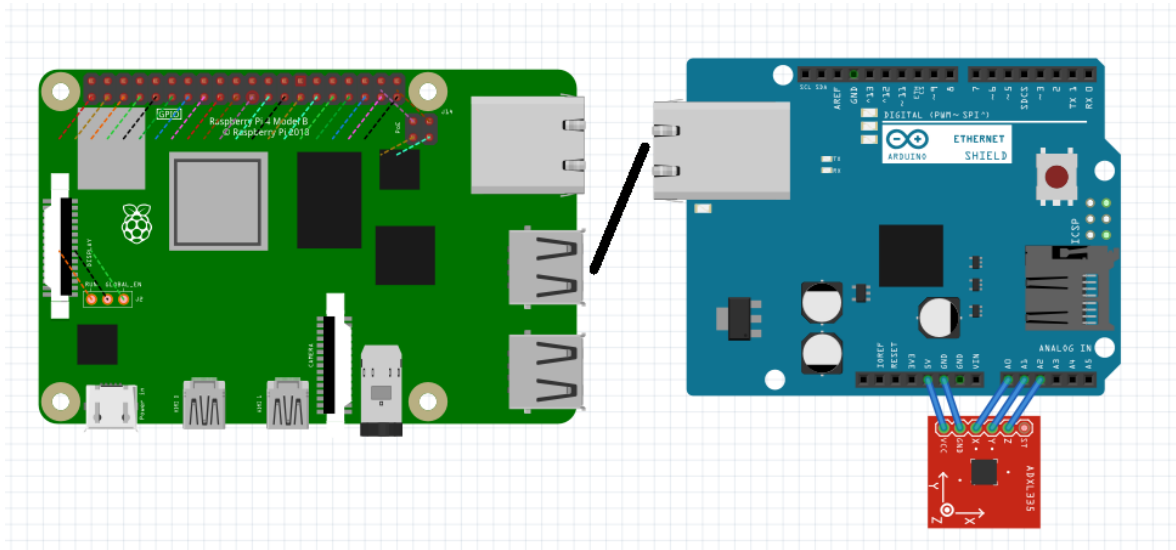


Figure 3.4: Hardware architecture with signal flow.

plemented through environment variables and Poetry dependency management. The system transmitted data in JSON format via HTTPS POST requests with a maximum payload size of 1 MB at 30-second intervals, incorporating timestamps, sensor IDs, and checksums for data integrity verification, following the structured payload schema defined in the API implementation.

Network performance exceeded all requirements with a **100 percent** transmission success rate across all API calls, zero timeout incidents, and no security breaches detected. This performance validates the robust error handling and logging mechanisms implemented in the HTTPS communication methodology. Each successfully transmitted batch contained a payload of approximately 12,847 data points—the length of the FFT result array—and this operation was repeated multiple times throughout the testing and deployment phase. Complete router specifications are provided in Appendix .5.

3.3 System Requirements

3.3.1 Functional Requirements

In addition to the aforementioned design traits of the system as a whole, there is also some design requirements related to the measuring limits of the sensor and communication among different components.

To begin with, the most commonly rotating speed of machinery goes up to 1500 [rpm]

corresponds to 25 [Hz]. This relationship is defined by the following equation:

$$f \text{ [Hz]} = \frac{r \text{ [RPM]}}{60} \quad (3.1)$$

The aforementioned relationship entails that the rotational speed of a machine can be interpreted into frequency, which in turn is of great help when related to Fourier Transform applications.

All in all, the functional requirements are highlighted hereunder:

- **Frequency Range:** 0–300 Hz (covers common machine faults like imbalance, misalignment).
- **Real-Time Processing:** On-the-fly FFT computation for immediate fault detection.
- **Scalability:** Modular design for multi-sensor deployments.

3.3.2 Non-Functional Requirements

- **Cost:** Minimize BOM cost (justifies Arduino + RPi4 over industrial PLCs).
- **Power:** Optimize for continuous operation (e.g., no active cooling).
- **Accuracy:** ADXL335's $\pm 3g$ range suffices for industrial vibrations (cite datasheet).

3.3.3 Network Requirements

The network requirements for the IoT sensor system were defined to ensure secure, reliable, and efficient transmission of data to the Supabase cloud database. The design considered the characteristics of the sensor payload, network infrastructure, and API constraints to optimize performance and maintain data integrity. Each data batch consisted of approximately 12,847 measurements in JSON format, with payload sizes ranging from 50 KB to 200 KB per transmission.

The system used a local router at IP 192.168.2.1 with MAC address 94:e3:ee:f0:20:bc, connected via a 2.4 GHz Wi-Fi band on channel 11. The Wi-Fi network utilized WPA2-Personal (AES/CCMP) encryption to secure local traffic.

With a 10 Mbps internet connection (1.25 MB/s), the network had sufficient bandwidth to handle sensor data uploads reliably. The Wi-Fi link speed (16.25 MB/s) ensured that local transmissions were fast and would not create a bottleneck for the payloads.

To ensure reliability and avoid Supabase API rate limits, each POST request was kept under 1 MB. Batching of multiple measurements per request optimized network utilization while preserving data accuracy. High signal strength and low packet loss guaranteed consistent transmission. Each payload included timestamps, sensor IDs, and optional checksums for integrity verification. Retry mechanisms and local buffering were implemented to recover from temporary network outages, ensuring zero data loss.

All data transmissions used HTTPS with TLS 1.3 encryption, and API key authentication was enforced. Local traffic was protected by WPA2. Optional signing or checksum verification of payloads further enhanced security against tampering.

The network infrastructure and transmission logic were designed to accommodate additional sensors, higher sampling rates, or larger payloads without compromising performance, reliability, or security. Overall, the combination of strong Wi-Fi signal, robust security protocols, and careful payload management ensured that the network could reliably support continuous, secure, and accurate transmission of IoT data to the Supabase cloud database.

Chapter 4

Implementation

4.1 Implementation Introduction

The implementation of the aforementioned system concept requires a study of the fundamental limitations of its individual components.

A primary potential bottleneck is the hardware serial transmit buffer of the Arduino Uno. For this project, the maximum capacity of the UART interconnection between the Arduino Uno and the Raspberry Pi 4 is a critical design parameter (the connection setup is detailed in Chapter 3). The Arduino's operation is defined by a batch-processing strategy: it must first read and store data from the ADXL335 accelerometer in its internal memory. Only after a predetermined amount of data is stored is the entire dataset transmitted to the Raspberry Pi 4. Figures 3.1 and 3.4 provide a clear visualization of this dataflow.

Subsequently, the Raspberry Pi 4 receives, aggregates, and processes this dataset. Its computational role is to execute a Fast Fourier Transform (FFT) and calculate the signal magnitude for analysis, translating the real-world time-domain data into the frequency domain (see Chapter 2). And the processed results are sent to a database for storage and further use.

Following the frequency-domain analysis, the Raspberry Pi 4 executes the final stage of the data pipeline by transmitting the processed results to a cloud-based PostgreSQL database hosted on Supabase. This is achieved by leveraging Supabase's auto-generated RESTful API, which provides a secure and programmatic interface for data operations. The Pi utilizes HTTPS requests, specifically authenticated POST calls, to insert structured JSON payloads containing the computed FFT magnitudes and frequency bins directly into the designated tables within the Supabase schema. This serverless architecture eliminates the need for man-

aging database infrastructure, while the built-in row-level security (RLS) ensures that data access is strictly governed by predefined policies. By interfacing with the Supabase API, the system ensures that analyzed sensor data is persistently stored in a scalable and readily queryable data store, facilitating further visualization, historical analysis, or integration with other applications within the Supabase ecosystem.

A flowchart that illustrates a brief dataflow is depicted on Figure 4.1.

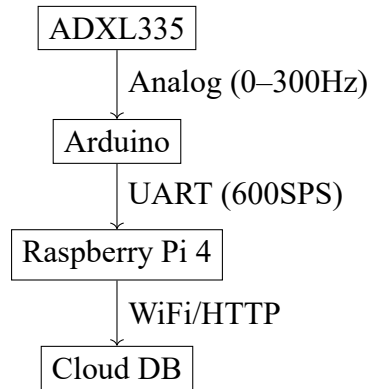


Figure 4.1: End-to-end data flow with protocols

More technical details on the aforementioned topics are briefly analyzed within the present chapter.

4.2 Software Architecture and Data Flow

4.2.1 The Arduino UNO Algorithm

The main focus of the Arduino UNO sketch is to implement a batched data processing strategy that enables sequential transmission of sensor data to the Raspberry Pi 4. This design methodology specifically addresses the memory constraints of the Arduino UNO by minimizing the instantaneous SRAM utilization through carefully sized data batching.

The Arduino UNO receives analog signals from the sensor breakout board through its integrated analog-to-digital converter (ADC), which features 10-bit resolution across six input channels. Each designated analog input pin (A0, A1, A2) continuously monitors voltage levels in the range of 0,V to 5,V, corresponding to the physical phenomena being measured by the sensors, which entails that the reading can be interpreted as g's or mm/s². The ADC performs successive approximation conversion at a sampling rate of approximately 10,000 samples per second, transforming continuous analog voltages into discrete digital values ranging from 0

(representing 0V) to 1023 (representing 5V). Each converted value is stored as a 16-bit signed integer (2 bytes) in memory, maintaining compatibility with the Arduino's architecture while preserving measurement precision.

Algorithm 1 Snippet of Data Acquisition & Transmission (see the whole code on Appendix Section 2.1)

Require: $numChannels = 3$, $analogPins = [A0, A1, A2]$, $bufferSize = 10$

Ensure: Serial transmission of analog data

```

1: Initialize  $analogBuffer[numChannels][bufferSize]$ ,  $tempVal \leftarrow 0$ 
2: while true do
3:   for  $j = 0$  to  $bufferSize - 1$  do
4:     for  $i = 0$  to  $numChannels - 1$  do
5:        $analogBuffer[i][j] \leftarrow analogRead(analogPins[i])$ 
6:     end for
7:   end for
8:   if  $tempVal = 100$  then
9:      $tempVal \leftarrow 0$  {Calibration}
10:  else
11:    for  $j = 0$  to  $bufferSize - 1$  do
12:      for  $i = 0$  to  $numChannels - 1$  do
13:         $Serial.print(analogBuffer[i][j])$ 
14:        if  $i < numChannels - 1$  then
15:           $Serial.print(", ")$ 
16:        end if
17:      end for
18:       $Serial.println()$ 
19:    end for
20:     $tempVal \leftarrow tempVal + 1$ 
21:  end if
22: end while

```

The data acquisition process follows a structured double-buffering approach where readings are temporarily stored in a two-dimensional array before processing and transmission. This architecture ensures temporal consistency between related sensor measurements while

optimizing memory utilization through fixed-size allocation.

A snippet of the Arduino UNO algorithm implementation follows hereunder (Algorithm 1):

The memory requirement in this case can be estimated , combining all received data as analog signal by the sensor breakout:

$$\text{Total Memory} = (\text{numChannels} \times \text{bufferSize} \times \text{sizeof(int)}) + \text{Overhead} = 3 \times 10 \times 2 = 60$$

The core data storage therefore occupies exactly 60 bytes for sensor readings. This buffer size was determined through extensive empirical testing and validation using methodical trial-and-error optimization. Increasing the buffer size beyond this optimal value consistently resulted in system instability and memory allocation failures, while smaller buffers reduced data throughput efficiency.

Although 60 bytes appears minimal compared to the Arduino UNO's 2048 bytes of SRAM and 32 KB of flash memory (as extensively discussed in Chapter 3), several critical memory overhead factors must be considered. The hardware serial communication infrastructure requires 128 bytes of dedicated SRAM space, comprising dual 64-byte circular buffers for transmit (TX) and receive (RX) operations. Additional memory is consumed by program stack operations, interrupt service routines, and the Arduino core libraries, which typically utilize 200-300 bytes during normal operation.

Furthermore, the script incorporates several control variables and system parameters that contribute to cumulative memory consumption. These include loop counters, temporary variables, and the program instruction pointer, which collectively add approximately 20-30 bytes of overhead. This memory fragmentation, combined with the inherent memory management characteristics of the AVR architecture, creates a persistent risk of stack-heap collisions and memory exhaustion scenarios. The implementation therefore employs conservative memory practices, including static allocation and avoidance of dynamic memory operations, to mitigate these risks and ensure system reliability during extended operation periods.

The chosen architecture represents an optimal balance between data throughput requirements and memory constraints, ensuring reliable operation while maximizing the available resources for future expansions or additional functionality.

The FFT is applied to the acceleration data collected from the Arduino to analyze its frequency content. The data from all three axes (X, Y, Z) and the combined amplitude are

transformed from the time domain to the frequency domain. The actual sampling rate F_s is calculated from the timestamps of the collected data, and the FFT is performed using a configurable number of points L . The result is a single-sided amplitude spectrum for each signal. A larger F_s increases the maximum measurable frequency, while a larger L improves frequency resolution. The detailed steps of this process are summarized in Algorithm 2.

Algorithm 2 FFT Processing of Arduino Sensor Data

Require: Time-domain sensor data for X, Y, Z axes and combined amplitude, measurement timestamps, FFT length L

Ensure: Single-sided amplitude spectrum $P1$ and frequency vector f for each signal

- 1: Compute differences between successive timestamps
 - 2: Calculate sampling rate: $F_s \leftarrow 1/\text{average}(\text{time differences})$
 - 3: Choose number of FFT points L
 - 4: **for** each signal in [Amplitude, X, Y, Z] **do**
 - 5: Compute FFT of the signal with length L
 - 6: Compute magnitude and normalize by L
 - 7: Keep first half of the FFT (single-sided spectrum)
 - 8: Multiply all values except first and last by 2 to conserve energy
 - 9: Generate frequency vector: $f \leftarrow F_s/L \times \text{bin indices}$
 - 10: **end for**
-

4.2.2 The Raspberry Pi 4 Algorithm

While the Arduino UNO sketches seem to be quite straight forward and the only difficulty that had to be overcome was its memory limitations, the Raspberry Pi 4 algorithms are far more sophisticated.

An efficient way to organise the files of a project is the "src layout" [24]. A well-structured project layout, such as the src-based approach with clearly separated folders for source code, tests, configs, and scripts, is highly effective because it enforces organization and scalability as a project grows. By isolating source code under `src/`, you avoid common import conflicts and ensure your code behaves the same way in both development and production.

Configuration files and environment variables are separated from the core logic, which improves maintainability and allows developers to adapt the project across environments (local, staging, production) without modifying source code. This separation of concerns makes

collaboration easier, onboarding smoother, and scaling more manageable as features and complexity increase.

Within this setup, Poetry and `.env` files play crucial roles. Poetry simplifies dependency management by consolidating everything into a single `pyproject.toml`, handling versioning, virtual environments, and packaging in a reproducible way. This ensures consistency across machines and deployments, preventing “works on my machine” problems. On the other hand, `.env` files keep sensitive information (like API keys, database credentials, and tokens) out of the codebase, promoting security and flexibility. Paired with a `.env.example`, they also document required environment variables without leaking secrets. Together, Poetry and `.env` form the backbone of a professional, maintainable, and secure Python project setup.

The srclayout project structure looks like this:

```
vibration-monitoring-project/
├── .venv/                # Python virtual environment
├── config/
│   └── config.yaml      # Application configuration
├── sourceCode/
│   ├── main.py          # Main application
│   ├── api.py           # API communication
│   ├── data_processing.py # Signal processing
│   ├── serial_communication.py # Serial communication
│   └── utils.py         # Utility functions
├── testing/
│   └── test_methods.py   # Test cases
├── poetry.lock          # Dependency lock
└── .env                 # Environment variables
```

Algorithm 3 Raspberry Pi Data Processing Pipeline**Require:** Serial port *port*, baud rate *baud_rate***Ensure:** Processed vibration analysis results

```

1: Initialize serial connection: ser  $\leftarrow$  setup_serial_connection(port, baud_rate)
2: history  $\leftarrow$  [], historyX  $\leftarrow$  [], historyY  $\leftarrow$  [], historyZ  $\leftarrow$  []
3: start_time  $\leftarrow$  current_time(), time_window  $\leftarrow$  config['time_window']
4: while current_time() - start_time < time_window do
5:   timestamp  $\leftarrow$  current_time()
6:   x, y, z  $\leftarrow$  read_serial_data(ser)
7:   amplitude  $\leftarrow$  calculate_amplitude(x, y, z)
8:   Append values to history, historyX, historyY, historyZ
9: end while
10: Fs  $\leftarrow$  calculate_sampling_rate(timestamps)
11: L  $\leftarrow$  Desiredlength {FFT points}
12: for each dataset in [history, historyX, historyY, historyZ] do
13:   spectrum, frequencies  $\leftarrow$  compute_fft(dataset, Fs, L)
14:   result  $\leftarrow$  detect_unbalancing(frequencies, rpm = 1800, spectrum, threshold =
      0.1)
15:   Send results to API: api.send_measurement(...)
16: end for
17: return Processing complete

```

An overview of the core processing workflow implemented in the project is presented in Algorithm 3, highlighting the principal computational processes and methodological practices employed. This snippet captures the essential data acquisition, processing, and transmission pipeline that forms the foundation of the vibration monitoring system.

For a comprehensive examination of the complete project architecture, readers are directed to the Appendix section. Specifically, Section 2.2 provides detailed algorithmic descriptions of all constituent scripts, including their functional specifications, inter-module communication protocols, and implementation details. The appendix offers complete visibility into the system's modular design, encompassing data acquisition, signal processing, API communication, and utility functions that collectively enable the real-time vibration monitoring capabilities.

The serial communication module, as previously highlighted in the system architecture, establishes a connection with the Arduino microcontroller at a specified baud rate, initializing the serial port with a timeout parameter to prevent blocking operations.

Specifically, the data received by Arduino UNO are getting analyzed. The sensors yields the 3-axis readings (X, Y and Z), when the whole dataset, for a predetermined time-window is received by Raspberry Pi, the amplitude of the 3-axis readings is calculated and afterwards frequency domain data are extracted.

Upon initialization, the Raspberry Pi waits until the Arduino Uno shares the sensor readings. The received data string, containing three comma-separated acceleration values from the sensor's X, Y, and Z axes, is decoded from UTF-8 format, stripped of extraneous characters, and parsed into floating-point numerical values representing the triaxial acceleration measurements in g-forces, which are subsequently returned for further processing in the vibration monitoring pipeline, consistent with the data flow described in earlier sections.

Moreover, the API communication for transmitting vibration data to the Supabase backend is implemented through a structured workflow that begins with authentication and configuration validation. Upon class instantiation, the system employs the `requests.post()` method to transmit JSON-formatted vibration data to the Supabase REST API endpoint, ensuring secure communication through properly configured HTTP headers containing authentication tokens and content type specifications.

The `requests.post()` command functions as the core mechanism for data transmission, operating as an HTTP client that serializes the vibration data payload into JSON format and dispatches it via POST request to the designated Supabase endpoint. This method automatically handles connection management, timeout protocols, and response processing while incorporating the pre-configured headers containing the essential API key for authentication. The implementation includes robust error handling that captures potential exceptions such as connection failures, timeout errors, or invalid response formats, while also validating the HTTP status codes returned by the server to ensure successful data ingestion. The method returns a response object containing the server's status code and optional response data, which the system then processes through comprehensive logging mechanisms for both debugging and operational monitoring purposes.

For each data transmission request, the system first resolves the sensor identifier by querying the Supabase database using the sensor name, establishing a relational link between the

measurement data and the specific sensor entity. The payload construction encapsulates critical metadata including the sensor ID, precise time intervals, measurement coordinate (X, Y, Z, or magnitude), and the actual FFT-processed values array. The HTTP POST request is executed with proper error handling and comprehensive logging, featuring response validation that checks for empty or malformed JSON responses while maintaining robust exception handling for network failures or authentication errors. The implementation includes diagnostic logging at multiple stages, providing visibility into the payload structure, response status codes, and JSON content for debugging and monitoring purposes, ensuring reliable data transmission to the cloud-based storage infrastructure.

Moreover it is worth mentioning that the development and testing environment spanned multiple hardware platforms, utilizing Poetry for consistent dependency management across all systems:

- **Raspberry Pi devices:** Model 4B (4GB RAM) running Raspberry Pi OS
- **Testing PCs:** Fedora 40 and Windows 11 systems

Poetry ensured identical dependency versions across all platforms through the `pyproject.toml` specification. Key packages maintained consistency included:

- **pyserial v3.5:** Uniform serial communication across platforms
- **numpy v1.24.3:** Consistent numerical computation
- **requests v2.28.2:** Standardized HTTP client functionality
- **python-dotenv v1.0.0:** Uniform environment variable handling

The cross-platform approach provided several advantages:

- Development flexibility across Windows and Linux environments
- Consistent behavior between development and deployment on Raspberry Pi
- Simplified testing across multiple operating systems
- Streamlined collaboration through version-locked dependencies

Testing covered all platform combinations to ensure compatibility and consistent performance across different devices. The environment successfully supported the complete development lifecycle from initial coding to final deployment on production hardware.

4.2.3 Vercel Implementation and Supabase Integration

After sending all data to Supabase, you can deploy this Next.js-friendly application that visualizes your sensor measurements. The project structure is optimized for Next.js with TypeScript and includes all necessary configuration files. When you deploy this application to Vercel, you'll receive a unique URL that you can share with specific email addresses.

Vercel is an online platform for hosting web applications and can be used to create dynamic pages that fetch and display real-time data. In your project, Vercel hosts a Next.js dashboard that retrieves sensor measurements from a Supabase API. Using classic client-side rendering (CSR), your React component calls `fetch('/api/measurements')` on the client, awaits the data, and then uses it to create plots. The charts are rendered only after the data arrives, allowing users to see the most recent measurements dynamically without requiring a page rebuild.

The dashboard is designed to compare current sensor readings with a stored healthy baseline for each machinery coordinate (X, Y, Z, Magnitude). The earliest measurement for each coordinate is considered the healthy state, while the latest measurement is treated as the current state. The data is processed in the client by grouping measurements and combining the healthy and current values for each frequency point, producing arrays that are then used for plotting. This structure allows for a clear visual comparison of baseline versus current vibration data for each axis.

Each plot displays two lines on the same chart: one representing the healthy baseline and the other representing the current measurements. This enables users to immediately identify deviations or anomalies in the machinery's performance. The system can be enhanced with deviation highlighting, real-time updates via Supabase Realtime, or server-side preprocessing for large datasets. Overall, this approach creates a fully dynamic, interactive monitoring dashboard hosted on Vercel that provides immediate visual feedback on machinery health.

The access control can be manipulated by configuring Vercel's authentication settings to restrict the deployment URL to only specific email addresses, ensuring your sensor data remains private while being easily accessible to authorized viewers.

These packages enabled secure communication with the Supabase backend while maintaining optimal performance in the Vercel environment. The application structure was organized as follows:

```
my-nextjs-project/
```

```
├─ .next/
├─ node_modules/
├─ public/
├─ src/
│  └─ app/
│     └─ api/
│        └─ route.ts
│        └─ fonts/
│        └─ lib/
│           └─ supabase.ts
│           └─ favicon.ico
│           └─ globals.css
│           └─ layout.tsx
│           └─ page.tsx
├─ .env.local
├─ .eslintrc.json
├─ tsconfig.json
├─ package.json
├─ package-lock.json
├─ next.config.ts
├─ next-env.d.ts
├─ tailwind.config.ts
├─ postcss.config.js
```

The most important algorithm that is being called is the `page.tsx` component, which serves as the main data visualization dashboard that fetches sensor measurements from the Supabase database through the API endpoint, processes the time-series data, and renders interactive charts displaying X, Y, Z axis readings along with magnitude values using the Recharts library for comprehensive sensor data analysis and monitoring.

This algorithm implements the core functionality of different end points. It fetches, processes and renders charts. More details on this can be looked upon the algorithms of the Subsection ??, while a snippet yields some intuition into the aforementioned practice (see

Algorithm 4).

Algorithm 4 Sensor Data Visualization Algorithm

```

1: Import React, Recharts components
2: Define Measurement type structure
3:
4: Function Home():
5:   Initialize state for measurements and error
6:   Fetch data from API endpoint on component mount
7:   Handle errors and loading states
8:   Combine X, Y, Z coordinate data of reference and current state
9:   Render interactive charts using Recharts of X , Y , Z and their magnitude separately
10: End Function
11:
12: API Function GET():
13:   Query Supabase database for measurements
14:   Return data as JSON response
15:   Handle database errors appropriately
16: End Function
  
```

The Vercel deployment platform was utilized to host a Next.js application that provides real-time visualization of sensor measurements. The implementation required several Supabase integration packages to ensure secure and efficient data handling:

- `@supabase/supabase-js`: The official JavaScript client for Supabase services (database, authentication, storage)
- `@supabase/ssr`: Server-side rendering support for Next.js applications (replacing the deprecated `@supabase/auth-helpers-nextjs`)

The component serves as the primary user interface for accessing and interpreting the collected sensor measurements, making it the central algorithm in the data visualization pipeline.

This software runs in real time keeping track of all measurements of the actual mechanism. It stores the measurements of the first run as a healthy-reference point and every other measurement will be compared to that healthy-state reference dataset. These historical data

can be critical so as to locate the critical frequencies and any potential peaks magnitude increase.

Chapter 5

Results

5.1 Introduction

First and foremost, comprehensive testing protocols were implemented to validate system functionality and reliability across all components. Performance metrics were tracked to identify potential delays, data loss, or software malfunctions.

As previously discussed in Chapter 4, concerns regarding data receiving and transmission integrity were addressed. The Arduino UNO's hardware architecture imposes specific constraints on data handling capabilities, with its 64-byte serial buffers limiting maximum amount of data input to approximately 600 samples (considering the set-up and the implementation of this very project, which ensues that this is the estimated amount of data based on the dataset format) per second when transmitting three-axis acceleration data. This necessitated careful design of the communication protocol to prevent buffer overflows while maintaining real-time monitoring capabilities.

In addition, further investigation is carried out considering the Raspberry Pi data handling capacity, since the latter one has to receive, store, process and post data, which activity is far more demanding.

For testing and deployment, the Raspberry Pi 4 maintained a continuous connection to a host PC within the same local network. This configuration utilized the Raspbian operating system—a lightweight, Debian-based Linux distribution optimized for embedded systems—running on the Raspberry Pi. Communication between the host PC and Raspberry Pi was established through SSH (Secure Shell protocol), which provided secure remote access and full terminal control over the Raspberry Pi. This setup enabled real-time monitoring, debug-

ging, and management of the data acquisition processes without requiring direct physical access to the embedded system.

5.2 System Verification and Validation

5.2.1 Arduino to Raspberry Pi Data Transmission

The system was tested across all stages of data transmission, with careful consideration of the Arduino's operational limits. Sensor readings were first collected by the Arduino, which processed the raw signals from each axis and calculated the amplitude. The Arduino successfully managed the incoming sensor data without loss or timing issues, operating well within its 64-byte buffer constraints at the configured 115200 baud rate.

The Arduino's competency in receiving and transmitting data can be evaluated using Algorithm 7: "Testing Results" in the Appendix (see Section 2.2). This algorithm allows us to measure the effective sampling performance of the Arduino under typical operating conditions.

The sampling rate of the Arduino reflects its ability to acquire data from sensors. As discussed in Chapter 4, memory and processing limitations make data handling complex. In our experiments, the sampling rate F_s was calculated from the timestamps of acquired data by taking the average interval between successive samples and inverting it. This produced an average sampling rate of approximately 700 Hz, reflecting the Arduino's hardware capabilities.

When performing a Fast Fourier Transform (FFT) on the acquired data, the frequency resolution depends on both the sampling rate F_s and the number of FFT points L :

$$\Delta f = \frac{F_s}{L}$$

The Arduino reads three analog channels and sends the data to the Raspberry Pi via serial communication at 115,200 baud. Each reading from a channel is converted to a number (0–1023) and sent as a line of text with comma separators and a newline.

With a buffer of 10 samples per channel, a full buffer send is roughly 180 bytes. At 115,200 baud, the serial connection can transmit about 11,520 bytes per second, which allows approximately 64 buffer sends per second. This corresponds to roughly 640 samples per second per channel.

A larger F_s increases the maximum measurable frequency, while a larger L improves frequency resolution. The limitation that we encountered here is the upper limit of 350 Hz in the frequency domain results, while the sampling rate is 700 Hz.

Over a 30-second measurement window, the Raspberry Pi is expected to receive about 19,200 samples per channel, or roughly 57,600 samples total across the three channels. The actual measured data of 20,684 samples per channel closely matches this estimate, confirming that the system reliably handles the expected data rate without loss. It can be concluded here that no matter the amount of data received, the implementation described in Section 4.2.1 ensures proper handling.

Subsequently, the Arduino transmitted these batches to the Raspberry Pi via serial communication using an optimized request-response protocol. In this synchronous communication pattern, the Arduino sends exactly one line of three comma-separated acceleration values at a time, effectively preventing buffer overflows while ensuring data integrity. The Raspberry Pi efficiently received, buffered, and organized the incoming data, demonstrating that the system could handle the expected data rate of approximately 600 samples per second without data loss.

It is worth noting that even when the reading window is increased, the Arduino handles data transmission efficiently, as it communicates readings through the serial port in batches of 10 values.

5.2.2 **Raspberry Pi to Supabase and System Performance**

Finally, the Raspberry Pi transmitted the data to the Supabase API. By comparing the readings collected on the Raspberry Pi with the data successfully posted to Supabase, it was confirmed that the entire pipeline—from sensor to cloud—worked reliably. The main limitation observed was internet speed; however, the Wi-Fi link itself was more than sufficient to handle local transfers without delay.

Overall, each component of the system—from sensor to Arduino, Arduino to Raspberry Pi, and Raspberry Pi to Supabase—was capable of managing the data load while maintaining data integrity, timing requirements, and reliability within the Arduino’s operational constraints of 9,000–10,000 bytes per second. This is also illustrated in Figure 5.1, where the raw data from the Arduino is 20,684, the data received by the Raspberry Pi is also 20,684, and the processed data of length 500 has been received by Supabase. This can be easily validated

using the ‘values’ column.

```

1 SELECT
2   values
3 FROM measurements
4 WHERE id = '10a5d331-faaa-4212-ad71-eb394d65cc71';
5
6

VASILIS@VASILIS:~/repos/Vasilisdi-DiplomaUTH_Arduino-Raspberry $ poetry run python sourceCode/main.py
The currently activated Python version 3.11.2 is not supported by the project (^3.11.3).
Trying to find and use a compatible version.
Using python3.12 (3.12.0)
2025-09-03 00:37:46,865 - INFO - Loading config from: /home/VASILIS/repos/Vasilisdi-DiplomaUTH_Arduino-Raspberry/config/config.
yaml
2025-09-03 00:37:49,246 - INFO - Loading config from: /home/VASILIS/repos/Vasilisdi-DiplomaUTH_Arduino-Raspberry/config/config.
yaml
raw data: Len=2068
Magnitude: Len=500
2025-09-03 00:38:23,489 - INFO - API Response Status Code: 201
2025-09-03 00:38:23,489 - INFO - API Response is empty (no JSON returned).
X: Len=500
2025-09-03 00:38:25,376 - INFO - API Response Status Code: 201
2025-09-03 00:38:25,376 - INFO - API Response is empty (no JSON returned).
Y: Len=500
2025-09-03 00:38:27,347 - INFO - API Response Status Code: 201
2025-09-03 00:38:27,347 - INFO - API Response is empty (no JSON returned).
Z: Len=500
2025-09-03 00:38:29,463 - INFO - API Response Status Code: 201
2025-09-03 00:38:29,463 - INFO - API Response is empty (no JSON returned).

```

Figure 5.1: Transmitting system operational validation.

Although verification of the data transmitted from the Arduino to the Raspberry Pi and ultimately posted to a Vercel dashboard was the most crucial part of this project, there is always the possibility of issues in other sections, such as Vercel’s data reception. For this reason, the amount of data received and posted by Vercel was also checked, and it was validated that the entire list of 500 data points was properly read.

The following tables relate to the performance of the Raspberry Pi, which can handle even larger amounts of data:

FFT Points	Runtime (s)
250	36
500	39
750	41
1000	43
1500	43
2500	44

Table 5.1: Runtime Raspberry Pi performance for different FFT output lengths

Time Window (s)	Rows of Raw Data	Runtime (s)
30	20,684	40
60	41,433	68
120	82,933	131

Table 5.2: Runtime Raspberry Pi performance for different time windows

The whole system was further checked for performance using cronjob commands, performing recursive data reading over an entire day. No temperature or similar failures of the electronics, microcomputer, or microcontroller were detected, confirming that the system operated reliably.

5.3 Data Depiction and Visualization

Prior to the system verification described in the previous section, testing was conducted using a simplified algorithm derived from the main implementation. These preliminary implementations can be found in Appendix 2.2 (Algorithm 1 and Algorithm 7).

The purpose of this algorithm was to validate initial data visualization, demonstrating that both raw time-domain signals and frequency-domain data could be successfully processed and simulated within the Python interface. This approach proved efficient during testing, as it enabled rapid verification of expected patterns. In particular, the frequency spectra produced by the FFT could be readily compared with patterns described in the relevant literature, serving as an early confirmation of correct signal behavior.

For the Vercel-based interface, visualization tools and interactive charts were employed

to depict the transmitted sensor data, supporting the analysis of amplitudes, frequency trends, and potential anomalies over time. Upon accessing the deployed webpage, the dashboard dynamically retrieves sensor data from Supabase, compares the current measurements against stored healthy baseline data, and plots both datasets on the same charts. This comparative visualization enables users to identify deviations at a glance, making the dashboard an effective, real-time monitoring tool deployed efficiently on Vercel.

The following figures illustrate the deployed Vercel-based dashboard used for vibration data monitoring. The interface presents a series of interactive plots that allow comparison between the stored healthy baseline state of the machinery and the latest current measurements retrieved from Supabase. Each chart provides a clear visual distinction between the two states, enabling the detection of deviations across the X, Y, and Z axes as well as the overall vibration magnitude. By scrolling through the interface, the user can review each coordinate in detail, observing the amplitude-frequency relationships that characterize both normal and potentially anomalous machine conditions.

It must also be noted that on each plot two lines are depicted: the blue line corresponds to the healthy state reference, while the green line illustrates the current state of the machinery.

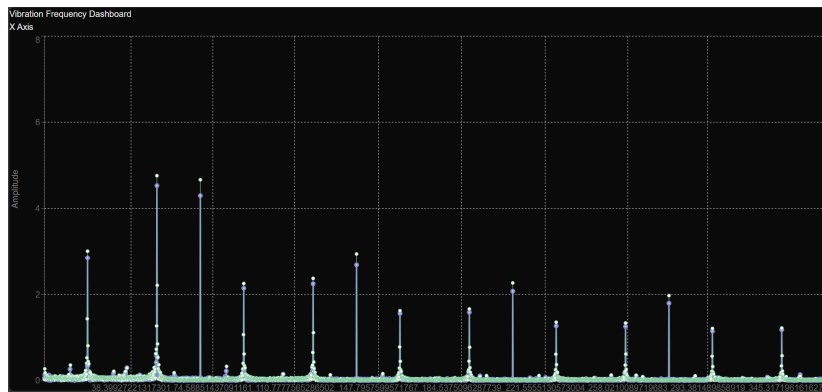


Figure 5.2: Visualization of the X-axis vibration data. The initial view of the Vercel dashboard shows the dynamically loaded measurement data from Supabase, where the healthy baseline (blue) is compared against the current state (green).

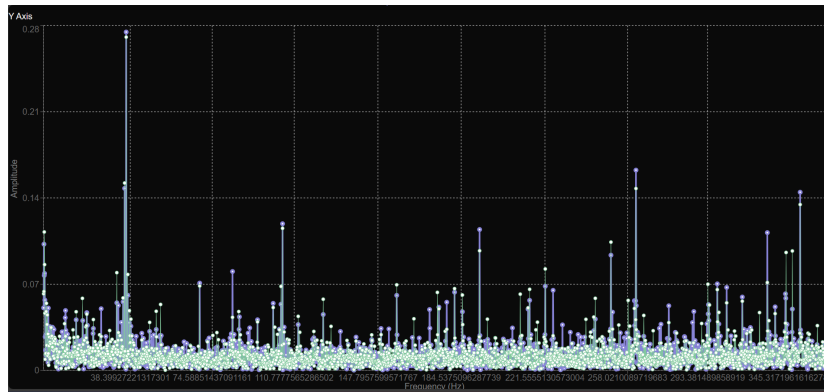


Figure 5.3: Visualization of the Y-axis vibration data. The chart compares the healthy base-line spectrum (blue reference) with the current sensor measurements (green), enabling the detection of frequency-domain deviations.

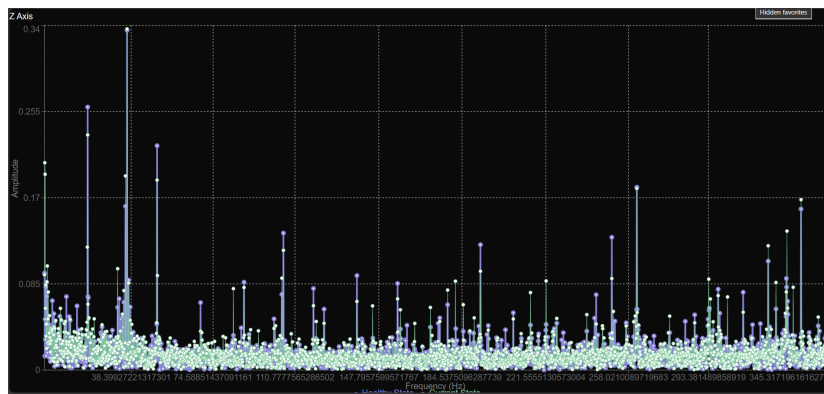


Figure 5.4: Visualization of the Z-axis vibration data. Similar to the X- and Y-axis plots, the frequency response is depicted with both the healthy baseline (blue) and current state (green), highlighting differences between conditions.

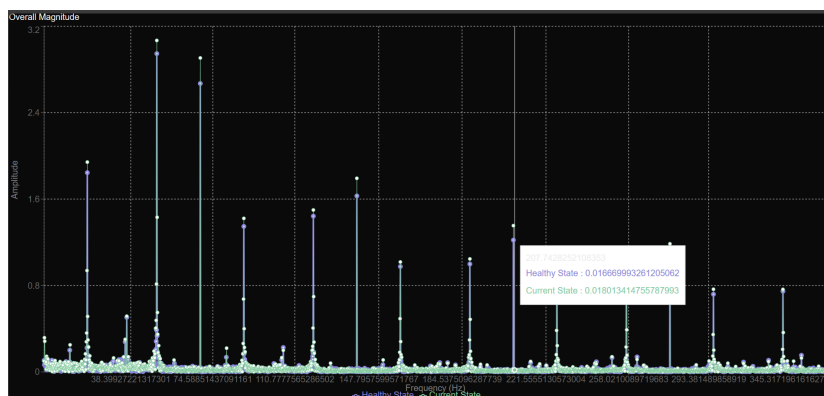


Figure 5.5: Visualization of the overall vibration magnitude. This chart combines all three co-ordinate axes into a single representation, showing the total vibration energy with the healthy baseline (blue) compared against the current state (green).

Chapter 6

Conclusions

This research has successfully designed, implemented, and validated a complete end-to-end system for machine condition monitoring with a specific focus on bearing defect detection through vibration analysis. The implemented solution effectively integrates hardware components including Arduino UNO, ADXL335 sensor, and Raspberry Pi 4 with software protocols and cloud services such as Supabase and Vercel, creating a reliable pipeline for data acquisition, processing, transmission, and visualization.

The fault identification methodology is based on comparative frequency analysis, where vibration spectra from operational machinery are compared against pre-established healthy baseline spectra. The system processes raw time-domain data through Fast Fourier Transform to generate frequency spectra, which are then visually presented on an interactive Vercel dashboard. The simultaneous display of both healthy baseline and current operational data across X, Y, Z axes and magnitude enables immediate visual identification of potential anomalies.

Data acquisition initiates at the sensor level with the ADXL335 accelerometer mounted on the target machinery and connected to the Arduino UNO microcontroller. The Arduino performs analog-to-digital conversion of triaxial acceleration signals and calculates the resultant magnitude. To address the inherent hardware limitations of the Arduino's 64-byte serial buffer, a sophisticated request-response protocol was developed and implemented. This protocol ensures the Arduino accumulates data in batches of 10 samples per axis and transmits comma-separated data lines containing X, Y, Z, and magnitude values only upon receiving explicit requests from the Raspberry Pi, maintaining a consistent sampling rate of approximately 700 Hz without data loss.

The analytical process is distributed across multiple stages within the system architec-

ture. Initial preprocessing occurs on the Arduino, where raw analog signals are converted to digital acceleration values and magnitude calculations are performed. Primary processing takes place on the Raspberry Pi, where time-domain data undergoes Fast Fourier Transformation to produce frequency-domain spectra essential for bearing fault diagnosis. The processed frequency data is subsequently structured and transmitted to Supabase, which serves as the central data repository for both historical baseline information and current measurements. The final analytical stage occurs on the Vercel dashboard, where frequency spectra are retrieved and presented through interactive visualizations. Although the current implementation requires human interpretation of spectral deviations, the infrastructure supports complete real-time data synchronization and availability.

6.1 Summary and Conclusions

This research has successfully achieved its primary objective of developing and validating a proof-of-concept condition monitoring system utilizing affordable, commercially available components. The implemented solution demonstrates that effective condition monitoring can be accomplished without relying on expensive proprietary systems, thereby significantly reducing the barriers to implementation for predictive maintenance applications. The system design effectively addresses critical embedded constraints, particularly the Arduino's limited buffer capacity, through an innovative communication protocol that ensures data integrity despite hardware limitations. Validation testing confirmed the system's operational reliability, with the Raspberry Pi consistently receiving and processing exactly 20,684 samples per channel during 30-second measurement windows. The deployment of a dynamic, real-time dashboard on Vercel provides an effective interface for operational monitoring and comparative analysis.

Several limitations and challenges were identified during system development and validation. The analytical upper frequency boundary is constrained by the Arduino's effective sampling rate, in accordance with Nyquist theorem requirements. The current system implementation necessitates human intervention for visual spectrum comparison and fault interpretation, introducing subjectivity and limiting scalability. Additionally, the comparative analysis methodology requires the availability of a verified healthy baseline spectrum for each specific machine under monitoring. This threshold is in essence the first run readings or

the healthy state readings.

It is also worth stressing that despite the limitations of the Arduino UNO R3 and Raspberry Pi 4, all potential bugs and system malfunctions have been overcome. This ensures that the way the whole project set-up was designed lead to great results and no issues during the operation simulation were noticed.

In conclusion, this thesis establishes that cost-effective IoT-based solutions can provide viable and efficient vibration-based fault detection capabilities. The research contributes a validated architectural blueprint for constructing such systems, addressing practical challenges related to data handling, serial communication, and full-stack deployment. The resulting implementation provides a solid foundation for developing automated, real-time condition monitoring solutions for industrial applications.

6.2 Future Research

The research presented in this thesis provides a comprehensive foundation for vibration data acquisition and visualization, yet the current dependence on visual comparison by human operators for identifying faults at characteristic frequencies represents a significant limitation. This manual interpretation approach introduces subjectivity, requires substantial time investment, and proves impractical for monitoring multiple machines simultaneously. Consequently, the most critical and logical progression for future research involves transitioning from a visualization platform to an automated diagnostic system.

The highest priority for subsequent investigation involves implementing automated fault detection and diagnosis through machine learning methodologies. This direction would require developing advanced feature extraction techniques to automatically identify key characteristics within frequency spectrum data, including amplitudes at specific characteristic fault frequencies—such as Ball Pass Frequency Outer race (BPFO), Ball Pass Frequency Inner race (BPFI), Ball Spin Frequency (BSF), and Fundamental Train Frequency (FTF)—as well as statistical features such as kurtosis, skewness, and root mean square values.

Further research should explore edge computing optimization to support on-device machine learning inference and reduce operational latency. Future work could migrate processing capabilities to more powerful microcontroller units such as the Arduino Nano 33 BLE Sense, featuring a Cortex-M4F processor with hardware floating-point unit support. This

architectural enhancement would enable complete Fast Fourier Transform and fault diagnosis pipelines to execute directly on edge devices, communicating only essential alerts and summary data to cloud services, thereby significantly improving system responsiveness and reliability.

Enhanced real-time capabilities and automated alerting mechanisms represent another promising research direction. Integrating Supabase's real-time functionality would enable instantaneous dashboard updates, while advanced alert systems could trigger automated notifications through email, SMS, or API calls to maintenance management systems immediately upon fault detection, effectively transforming the platform from a monitoring tool into a proactive maintenance alert system.

System expansion through multi-sensor data fusion would further improve diagnostic accuracy. Incorporating additional sensors for temperature monitoring and acoustic emission detection would enable the collection of multi-modal data streams. Sophisticated artificial intelligence models could then fuse these diverse data sources to provide more comprehensive and reliable machine health assessments while reducing false positive rates.

Finally, long-term validation through industrial deployment remains essential for practical system evaluation. Extended operational deployment in authentic industrial environments would facilitate the collection of validated fault datasets, enable continuous refinement of machine learning models, and provide crucial evidence regarding the system's economic value and reliability within actual predictive maintenance schedules. This transition from laboratory validation to industrial implementation represents the necessary evolution from proof-of-concept to practical tooling for industry applications.

Bibliography

- [1] Heinz P. Bloch. Maintenance cost reduction. In Heinz P. Bloch, editor, *Improving Machinery Reliability*, volume 1 of *Lecture Notes in Computer Science*, pages 434–484. Gulf Professional Publishing, 1998.
- [2] Bearing fault detection using vibration analysis. <https://ncd.io/blog/bearing-fault-detection-vibration-analysis/>. Accessed: 02-06-2025.
- [3] Zhiqiang Chen, Xudong Chen, Chuan Li, René Vinicio Sanchez, and Huafeng Qin. Vibration-based gearbox fault diagnosis using deep neural networks. *Journal of Vibro-engineering*, 19:2475–2496, 2017.
- [4] Giovanni Betta, Consolatina Liguori, Alfredo Paolillo, and Antonio Pietrosanto. A dsp-based fft-analyzer for the fault diagnosis of rotating machine based on vibration analysis. *IEEE Transactions on Instrumentation and Measurement*, 51:1316–1321, 12 2002.
- [5] Gangani Dharmarathne, A. M.S.R. Abekoon, Madhusa Bogahawaththa, Janaka Alawatugoda, and D. P.P. Meddage. A review of machine learning and internet-of-things on the water quality assessment: Methods, applications and future trends. *Results in Engineering*, 26, 6 2025.
- [6] Luigi Atzori, Antonio Iera, and Giacomo Morabito. The internet of things: A survey. *Computer Networks*, 54:2787–2805, 10 2010.
- [7] Liakot Ali, Roslina Sidek, Ishak Aris, Alauddin Mohd. Ali, and Bambang Sunaryo Suparjo. Design of a micro-uart for soc application. *Computers and Electrical Engineering*, 30:257–268, 6 2004.

- [8] About raspberry pi 4. <https://www.raspberrypi.com/>. Ημερομηνία πρόσβασης: 07-08-2025.
- [9] Michael S. Hamada, Alyson G. Wilson, C. Shane Reese, and Harry F. Martz. Bayesian reliability. *Springer New York, NY*, 1:143–159, 7 2001.
- [10] Patrick Ruane, Patrick Walsh, and John Cosgrove. Validation of a digital simulation model for maintenance in a high-volume automated manufacturing facility. In *5th IFAC Workshop on Advanced Maintenance Engineering, Services and Technologies AMEST 2022*, volume 55, pages 127–132. Elsevier B.V., 7 2022.
- [11] Jerry G. Lilly, William D. Marscher, Roger J. Cronin, Ray A. Gall, Richard O. Garbus, Willard O. Keightley, J. Davis Miller, Aloysius M. Mocek, William E. Nelson, Randall R. Parks, Jerry P. Pollack, James W. Schettler, and Theodore B. Whiton. *Vibration and Noise*, pages 22.1–22.65. Butterworth-Heinemann, 3 edition, 1 2008.
- [12] C.J. Li and K. McKee. *BEARING DIAGNOSTICS*, pages 143–152. Elsevier, 1 2001.
- [13] Weidong Jiao, Zhijing Huang, and Yonghua Jiang. Fault diagnosis of gearboxes using feature extraction in time domain. In *2017 9th International Conference on Modelling, Identification and Control (ICMIC)*, pages 573–577, 2017.
- [14] Victor Girondin, Herve Morel, Jean Philippe Cassar, and Komi Midzodzi Pekpe. Vibration-based fault detection of sharp bearing faults in helicopters. *IFAC Proceedings Volumes*, 45:180–185, 1 2012.
- [15] Mohamad Hazwan Mohd Ghazali and Wan Rahiman. Vibration-based fault detection in drone using artificial intelligence. *IEEE Sensors Journal*, 22:8439–8448, 5 2022.
- [16] Μαρία Κωνσταντινίδου-Σεραφειμίδου. *Διαφορικές Εξισώσεις*. Σοφία, Θεσσαλονίκη, 3 edition, 2009.
- [17] James W. Cooley and John W. Tukey. An algorithm for the machine calculation of complex fourier series. *Mathematics of Computation*, 19(90):297–301, 1965.
- [18] Richard C. Singleton. On computing the fast fourier transform. *Communications of the ACM*, 10(10):647–654, Oct. 1967.

- [19] Discrete fourier transform. <https://numpy.org/doc/2.2/reference/routines.fft.html>. Ημερομηνία πρόσβασης: 02-06-2025.
- [20] Jayavardhana Gubbi, Rajkumar Buyya, Slaven Marusic, and Marimuthu Palaniswami. Internet of things (iot): A vision, architectural elements, and future directions. *Future Generation Computer Systems*, 29:1645–1660, 2013.
- [21] Adxl335 accelerometer breakout. <https://www.analog.com/en/products/adxl335.html>, 2023. Accessed: September 5, 2025.
- [22] Analog input. <https://docs.arduino.cc/learn/microcontrollers/analog-input/>. Ημερομηνία πρόσβασης: 06-07-2025.
- [23] About arduino. <https://docs.arduino.cc/resources/datasheets/A000066-datasheet.pdf/>. Ημερομηνία πρόσβασης: 07-08-2025.
- [24] About organising projects in python. <https://packaging.python.org/en/latest/discussions/src-layout-vs-flat-layout/>. Ημερομηνία πρόσβασης: 07-08-2025.

APPENDICES

Appendix

Τίτλος Παραρτήματος

1 Tables

Table .1: Analog Pins (JANALOG)

Pin	Function	Type	Description
1	NC	NC	Not connected
2	IOREF	IOREF	Reference for digital logic V (connected to 5V)
3	Reset	Reset	Reset
4	+3V3	Power	+3.3V Power Rail
5	+5V	Power	+5V Power Rail
6	GND	Power	Ground
7	GND	Power	Ground
8	VIN	Power	Voltage Input
9	A0	Analog/GPIO	Analog input 0 / GPIO
10	A1	Analog/GPIO	Analog input 1 / GPIO
11	A2	Analog/GPIO	Analog input 2 / GPIO
12	A3	Analog/GPIO	Analog input 3 / GPIO
13	A4/SDA	Analog input/I2C	Analog input 4 / I2C Data line
14	A5/SCL	Analog input/I2C	Analog input 5 / I2C Clock line

Table .2: Digital Pins (JDIGITAL)

Pin	Function	Type	Description
1	D0	Digital/GPIO	Digital pin 0 / GPIO
2	D1	Digital/GPIO	Digital pin 1 / GPIO
3	D2	Digital/GPIO	Digital pin 2 / GPIO
4	D3	Digital/GPIO	Digital pin 3 / GPIO
5	D4	Digital/GPIO	Digital pin 4 / GPIO
6	D5	Digital/GPIO	Digital pin 5 / GPIO
7	D6	Digital/GPIO	Digital pin 6 / GPIO
8	D7	Digital/GPIO	Digital pin 7 / GPIO
9	D8	Digital/GPIO	Digital pin 8 / GPIO
10	D9	Digital/GPIO	Digital pin 9 / GPIO
11	SS	Digital	SPI Chip Select
12	MOSI	Digital	SPI Main Out Secondary In
13	MISO	Digital	SPI Main In Secondary Out
14	SCK	Digital	SPI Serial Clock Output
15	GND	Power	Ground
16	AREF	Digital	Analog Reference Voltage
17	A4/SD4	Digital	Analog input 4 / I2C Data line (duplicated)
18	A5/SD5	Digital	Analog input 5 / I2C Clock line (duplicated)

Table .3: Complete GPIO Pinout for Raspberry Pi 4 Model B (Part 1/2)

Pin #	Function	Alt Function	Voltage	Notes
1	3.3V	—	3.3V	50mA max
2	5V	—	5V	Power supply
3	GPIO2	SDA1	3.3V	I ² C Data
4	5V	—	5V	Power supply
5	GPIO3	SCL1	3.3V	I ² C Clock
6	GND	—	—	Ground
7	GPIO4	GPCLK0	3.3V	General purpose
8	GPIO14	TXD0	3.3V	UART Transmit
9	GND	—	—	Ground
10	GPIO15	RXD0	3.3V	UART Receive
11	GPIO17	—	3.3V	General purpose
12	GPIO18	PCM_CLK	3.3V	PWM0
13	GPIO27	—	3.3V	General purpose
14	GND	—	—	Ground
15	GPIO22	—	3.3V	General purpose

Table .4: Complete GPIO Pinout for Raspberry Pi 4 Model B (Part 2/2) (Continued from Table .3)

Pin #	Function	Alt Function	Voltage	Notes
16	GPIO23	–	3.3V	General purpose
17	3.3V	–	3.3V	50mA max
18	GPIO24	–	3.3V	General purpose
19	GPIO10	MOSI0	3.3V	SPI0
20	GND	–	–	Ground
21	GPIO9	MISO0	3.3V	SPI0
22	GPIO25	-	3.3V	General purpose
23	GPIO11	SCLK0	3.3V	SPI0
24	GPIO8	CE0	3.3V	SPI0 Chip Enable 0
25	GND	-	-	-
26	GPIO7	CE1	3.3V	SPI0 Chip Enable 1
27	ID_SD	-	-	EEPROM ID
28	ID_SC	-	-	EEPROM Clock
29	GPIO5	-	3.3V	General purpose
30	GND	-	-	-
31	GPIO6	-	3.3V	General purpose
32	GPIO12	PWM0	3.3V	PWM channel 0
33	GPIO13	PWM1	3.3V	PWM channel 1
34	GND	-	-	-
35	GPIO19	PCM_FS	3.3V	PWM/MISO1
36	GPIO16	-	3.3V	General purpose
37	GPIO26	-	3.3V	General purpose
38	GPIO20	MOSI1	3.3V	SPI1
39	GND	-	-	-
40	GPIO21	MISO1	3.3V	SPI1

Table .5: Router and Network Configuration Specifications

#	Property	Value	Description
1	Router IP (Gateway)	192.168.2.1	Local private IP address serving as network gateway
2	Operating Band	2.4 GHz	Longer range frequency, suitable for IoT applications
3	Channel	11	Optimized channel selection to minimize interference
4	Security Protocol	WPA2-Personal (AES/CCMP)	Enterprise-grade encryption for data protection
5	Signal Strength (RSSI)	93% / -42 dBm	Excellent connection quality with minimal packet loss
6	Link Speed	130 Mbps	More than adequate bandwidth for sensor data transmission
7	Internet Bandwidth	10 Mbps	Sufficient for external data transmission requirements

2 Algorithms

2.1 Arduino Data Acquisition Algorithm

Algorithm 1: Data Acquisition and Transmission System

```

1: Initialization:
2: numChannels = 3 {Number of analog channels}
3: analogPins = [A0, A1, A2] {Analog input pins}
4: bufferSize = 10 {Buffer size per channel}
5: analogBuffer = array[3][10] {Data storage array}
6: bufferIndex = array[3] {Buffer indices array}
7: tempVal = 0 {Control variable}
8:
9: Initialize serial communication at 115200 baud
10: for  $i = 0$  to numChannels - 1 do
11:   bufferIndex[ $i$ ]  $\leftarrow 0$  {Initialize buffer index}
12: end for
13:
14: while true do
15:   Phase 1: Data Acquisition and Storage
16:   for  $j = 0$  to bufferSize - 1 do
17:     for  $i = 0$  to numChannels - 1 do
18:        $xRaw = \text{analogRead}(\text{analogPins}[i])$ 
19:       analogBuffer[ $i$ ][ $j$ ] =  $xRaw$ 
20:     end for
21:   end for
22:
23:   Phase 2: Processing and Transmission
24:   if tempVal = 100 then
25:     Execute calibration procedures
26:     tempVal = 0
27:   else
28:     for  $j = 0$  to bufferSize - 1 do

```

```
29:      for  $i = 0$  to numChannels - 1 do
30:          Serial.print(analogBuffer[i][j])
31:          if  $i < \text{numChannels} - 1$  then
32:              Serial.print(, )
33:          end if
34:      end for
35:      Serial.println() {End of data line}
36:  end for
37:      tempVal = tempVal + 1
38:  end if
39: end while
```

2.2 Raspberry Pi Algorithms

Algorithm 1: main.py Implementation

```

1: from datetime import datetime, timedelta, timezone
2: from api import VibrationMonitoringAPI
3: from data_processing import calculate_amplitude, compute_fft
4: from serial_communication import setup_serial_connection, read_serial_data
5: from utils import get_api_key, load_config
6: import numpy as np
7:
8: # Load configuration
9: config = load_config()
10: # Load API key from environment variable
11: api_key = get_api_key()
12: if not api_key then
13:     raise ValueError("API key not found. Please set the API_KEY environment variable.")
14: end if
15: vibration_api = VibrationMonitoringAPI()
16: baud_rate = config['serial']['baud_rate']
17: port = config['serial']['port_linux']
18: # port = config['serial']['port_windows']
19: ser = setup_serial_connection(port, baud_rate)
20: history = []
21: history_X = []
22: history_Y = []
23: history_Z = []
24: time_window = config['time_window']
25: timestamps = []
26: start_time = datetime.now(timezone.utc)
27: time_delta = timedelta(seconds = time_window)
28:
29: try:
30:     while datetime.now(timezone.utc) - start_time < time_delta do

```

```
31: timestamp = datetime.now(timezone.utc)
32: timestamps.append(timestamp.timestamp())
33: col_1, col_2, col_3 = read_serial_data(ser)
34: amplitude = calculate_amplitude(col_1, col_2, col_3)
35: history.append(amplitude)
36: history_X.append(col_1)
37: history_Y.append(col_2)
38: history_Z.append(col_3)
39: end while
40: # Calculate the actual sampling rate
41: time_diffs = np.diff(timestamps)
42: Fs = 1/np.mean(time_diffs)
43: L = 1000 # number of FFT points
44: for  $i = 0$  to 3 do
45:     values, f = compute_fft(data, Fs, L)
46:     titles = [
47:         "Single-Sided Amplitude Spectrum of  $\sqrt{X(t)^2 + Y(t)^2 + Z(t)^2}$ ",
48:         "Single-Sided Amplitude Spectrum of  $X(t)$ ",
49:         "Single-Sided Amplitude Spectrum of  $Y(t)$ ",
50:         "Single-Sided Amplitude Spectrum of  $Z(t)$ "
51:     ]
52:     ylabels = ["|P1(f)|", "|P1X(f)|", "|P1Y(f)|", "|P1Z(f)|"]
53:     coordinate = ["Magnitude", "X", "Y", "Z"]
54:     api = VibrationMonitoringAPI()
55:     status_code = api.send_measurement("Test", coordinate[i], values.tolist(), start_time, timestamp)
56: end for
57: except KeyboardInterrupt:
58:     ser.close()
```

Algorithm 2: api.py Implementation

```

1: import requests
2: import json
3: import logging
4: from utils import load_config, get_api_key, get_supabase_url_posting, get_supabase_url_selecting

5: from datetime import datetime, timezone
6: from zoneinfo import ZoneInfo
7: config = load_config()
8:
9: class VibrationMonitoringAPI:
10:     def __init__(self):
11:         # Load API key and Supabase URL using the helper functions
12:         self.api_key = get_api_key()
13:         self.purl = get_supabase_url_posting()
14:         self.surl = get_supabase_url_selecting()
15:         # Ensure both the API key and Supabase URL are available
16:         if not self.api_key:
17:             logging.error("API Key is missing.")
18:             raise ValueError("API Key is required for the API.")
19:         if not self.purl:
20:             logging.error("Supabase URL is missing.")
21:             raise ValueError("Supabase URL is required for the API.")
22:         if not self.surl:
23:             logging.error("Supabase URL is missing.")
24:             raise ValueError("Supabase URL is required for the API.")
25:         # Initialize headers using the loaded API key
26:         self.headers = {
27:             'accept': 'application/json',
28:             'apikey': self.api_key,
29:             'Content-Type': 'application/json'
30:         }

```

```

31:         # Define the Athens time zone using ZoneInfo
32:         self.athens_tz = ZoneInfo('Europe/Athens')
33:     def get_time(self, start_measurement, end_measurement):
34:         # Convert the start and end times to Athens time zone
35:         start_measurement_athens = start_measurement.astimezone(self.athens_tz)
36:         end_measurement_athens = end_measurement.astimezone(self.athens_tz)
37:         # Format the timestamps in the required format with Athens time zone
38:         start_measurement_str = start_measurement_athens.strftime('%Y-%m-%dT%H :
%M : %S.%f')[:-3] + ' Z'
39:         end_measurement_str = end_measurement_athens.strftime('%Y-%m-%dT%H :
%M : %S.%f')[:-3] + ' Z'
40:         return start_measurement_str, end_measurement_str # end_measurement is now is not specified
41:
42:     def send_measurement(self, sensor_name, coordinate, values, start_measurement =
datetime.now(timezone.utc), end_measurement = datetime.now(timezone.utc)) :
43:         start_measurement_str, end_measurement_str = self.get_time(start_measurement, end_measurement)
44:         # Lookup sensor_id from sensor_name
45:         sensor_id = self.get_sensor_id(sensor_name)
46:         if not sensor_id:
47:             logging.error(f'Sensor with name '{sensor_name}' not found.')
48:             return None
49:         payload = {
50:             "sensor_id": sensor_id,
51:             "startMeasurement": start_measurement_str,
52:             "endMeasurement": end_measurement_str,
53:             "coordinate": coordinate,
54:             "values": values
55:         }
56:         response = requests.post(self.purl, headers = self.headers, data = json.dumps(payload))
57:         # Log the response status and data
58:         logging.info(f'API Response Status Code: {response.status_code}')
59:         #Check if response has content before parsing JSON

```

```
60:         if response.text.strip(): # Ensure response is not empty
61:             try:
62:                 logging.info(f"API Response JSON: " + str(response.json()))
63:             except requests.exceptions.JSONDecodeError:
64:                 logging.warning("API Response is not JSON (empty or invalid format).")
65:         else:
66:             logging.info("API Response is empty (no JSON returned).")
67:         return response.status_code
68:
69:     # New method to get sensor_id from sensor_name
70:     def get_sensor_id(self, sensor_name):
71:         query_url = f'"{self.surl}"?sensor_name=eq.{sensor_name}'
72:         response = requests.get(query_url, headers = self.headers)
73:         if response.status_code == 200 :
74:             data = response.json()
75:             if data:
76:                 return data[0]['id']
77:         else:
78:             logging.error(f"Failed to fetch sensor_id for sensor_name '{sensor_name}': {response.status}")
79:
80:     return None
```

Algorithm 3: data_processing.py Implementation

```

1: import numpy as np
2: import matplotlib.pyplot as plt
3: import math
4: import scipy.signal as signal
5:
6: def calculate_amplitude(col_1, col_2, col_3):
7:     return math.sqrt(col_1**2 + col_2**2 + col_3**2)
8:
9: def compute_fft(data, Fs, L):
10:     fft_result = np.fft.fft(data, L)
11:     P2 = np.abs(fft_result[1 :]/L)
12:     P1 = P2[: L//2]
13:     P1[1 : -1] = 2 * P1[1 : -1]
14:     return P1, Fs/L * np.arange(1, L//2 + 1)
15:
16: def compute_envelope(data):
17:     analytic_signal = signal.hilbert(data)
18:     envelope = np.abs(analytic_signal)
19:     return envelope
20:
21: def plot_fft(P1, f, title, ylabel):
22:     plt.figure()
23:     plt.plot(f, P1, linewidth = 3)
24:     plt.title(title)
25:     plt.xlabel( $f(Hz)$ )
26:     plt.ylabel(ylabel)
27:     plt.show()
28:
29: def unbalancing(frequency, rpm, spectrum, threshold):
30:     #index = f.index(rpm) to be used in main.py
31:     index = np.where((frequency > 0.999 * rpm)&(frequency < 1.001 * rpm))

```

```
32:  #print(index)
33:  frq = frequency[index]
34:  amplitude = spectrum[index]
35:  if amplitude > threshold :
36:      status = 'Shaft needs to be balanced'
37:  else:
38:      status = 'Balancing is ok'
39:  return f'RPM: {frq}, Amplitude: {amplitude}, Status: {status}'
```

Algorithm 4: serial_communication.py Implementation

```
1: import serial
2: import time
3:
4: def setup_serial_connection(port, baud_rate):
5:     ser = serial.Serial(port, baud_rate, timeout = 1.0)
6:     time.sleep(1)
7:     ser.reset_input_buffer()
8:     return ser
9:
10: def read_serial_data(ser):
11:     ser.write('rasp\n'.encode())
12:     while ser.in_waiting <= 0 :
13:         time.sleep(0.001)
14:     response = ser.readline().decode('utf - 8').rstrip()
15:     col_1, col_2, col_3 = map(float, response.split(','))
16:     return col_1, col_2, col_3
```

Algorithm 5: utils.py Implementation

```

1: import os
2: import yaml
3: import logging
4: from pathlib import Path
5: from dotenv import load_dotenv
6:
7: # Configure logging with timestamps
8: logging.basicConfig(
9:     level = logging.INFO,
10:    format = %(asctime)s - %(levelname)s - %(message)s,
11: )
12:
13: def load_env_variables():
14:     "Load environment variables from .env file."
15:     load_dotenv()
16:
17: def load_config(config_path = None):
18:     "Load YAML configuration file."
19:     if config_path is None:
20:         config_path = Path(__file__).resolve().parent.parent/'config'/'config.yaml'
21:     else:
22:         config_path = Path(config_path).resolve()
23:     logging.info(f'Loading config from: {config_path}')
24:     # Check if file exists
25:     if not config_path.is_file():
26:         raise FileNotFoundError(f'Config file not found at: {config_path}')
27:     try:
28:         with config_path.open('r') as file :
29:             # More readable pathlib usage
30:             config = yaml.safe_load(file)
31:     except yaml.YAMLError as e:

```

```
32:         raise ValueError(f'Error parsing YAML file at {config_path}: {e}')
33:     except Exception as e:
34:         raise RuntimeError(f'Unexpected error loading config: {e}')
35:     return config
36:
37: def get_api_key():
38:     """Retrieve API key from environment variables."""
39:     load_env_variables() # Ensure environment variables are loaded
40:     api_key = os.getenv('API_KEY')
41:     if not api_key:
42:         logging.warning("API_KEY is not set in the environment.")
43:         return None # Return None instead of raising an error
44:     return api_key
45:
46: def get_supabase_url_posting():
47:     """Retrieve Supabase URL from environment variables."""
48:     load_env_variables() # Ensure environment variables are loaded
49:     purl = os.getenv('SUPABASE_URL_FOR_POSTING')
50:     if not purl:
51:         logging.warning("SUPABASE_URL is not set in the environment.")
52:         return None # Return None if SUPABASE_URL is not set
53:     return purl
54:
55: def get_supabase_url_selecting():
56:     """Retrieve Supabase URL from environment variables."""
57:     load_env_variables() # Ensure environment variables are loaded
58:     surl = os.getenv('SUPABASE_URL_FOR_SELECTING')
59:     if not surl:
60:         logging.warning("SUPABASE_URL is not set in the environment.")
61:         return None # Return None if SUPABASE_URL is not set
62:     return surl
```

Algorithm 6: config.yaml Implementation

```
1: serial:
2:   # This applies in the case of Linux OS
3:   port_linux: "/dev/ttyACM0"
4:   # This applies in the case of Windows OS
5:   port_windows: "COM3"
6:   baud_rate: 115200
7: time_window: 30
8: # user defined
9: rpm: 300
10: bpfi: 3.4
11: bpfo: 2
12: ftf: 10
13: bsf: 5
14: threshold: 5
```

Algorithm 7: Testing Results

```
1: import time
2: import numpy as np
3: from sourceCode.utils import loadconfig
4: from sourceCode.dataprocessing import computeenvelope, computefft, calculateamplitude, plotfft, unl

5: from sourceCode.serialcommunication import setupserialconnection, readserialdata
6:
7: config = loadconfig()
8:
9: baudrate = config['serial']['baudrate']
10: port = config['serial']['portwindows']
11: rpm = config['rpm']
12: threshold = config['threshold']
13: ser = setupserialconnection(port, baudrate)
14:
15: time.sleep(1)
16: ser.resetinputbuffer
17:
18: history = []
19: historyX = []
20: historyY = []
21: historyZ = []
22: timewindow = config['timewindow']
23: timestamps = []
24: starttime = time.time()
25:
26: try:
27:     while time.time() - starttime < timewindow:
28:         timestamp = time.time()
29:         timestamps.append(timestamp)
30:         col1, col2, col3 = readserialdata(ser)
```

```
31:     amplitude = calculateamplitude(col1, col2, col3)
32:     history.append(amplitude)
33:     historyX.append(col1)
34:     historyY.append(col2)
35:     historyZ.append(col3)
36:
37:     timediffs = np.diff(timestamps)
38:     Fs = 1 / np.mean(timediffs)
39:
40:     L = 1000
41:     ylabels = ["|P1(f)|", "|P1X(f)|", "|P1Y(f)|", "|P1Z(f)|"]
42:
43:     for i, data in enumerate([history, historyX, historyY, historyZ]):
44:         P1, f = computefft(data, Fs, L)
45:         titles = [
46:             "Single-Sided Amplitude Spectrum of resultant acceleration",
47:             "Single-Sided Amplitude Spectrum of X-axis",
48:             "Single-Sided Amplitude Spectrum of Y-axis",
49:             "Single-Sided Amplitude Spectrum of Z-axis"
50:         ]
51:         print(unbalancing(f, rpm, P1, threshold))
52:         plotfft(P1, f, titles[i], ylabels[i])
53:
54:     for i, data in enumerate([history, historyX, historyY, historyZ]):
55:         Envelope = computeenvelope(data)
56:
57: except KeyboardInterrupt:
58:     ser.close()
```

2.3 Vercel Webpage Implementation

Vibration Frequency Data Visualization Dashboard Implementation

```
1: 'use client';
2: import { useEffect, useState } from 'react'
3: import { LineChart, Line, XAxis, YAxis, CartesianGrid, Tooltip,
  Legend, ResponsiveContainer } from 'recharts'
4: type Measurement = {
5:   id: string;
6:   startMeasurement: string;
7:   endMeasurement: string;
8:   coordinate: string;
9:   values: number[];
10:  frequencies: number[];
11: export default function Home() {
12:   const [measurements, setMeasurements] = useState<Measurement[]>([])
13:   const [error, setError] = useState<string | null>(null);
14:
15:   useEffect(() => {
16:     const fetchData = async () => {
17:       try
18:         const res = await fetch('/api/measurements');
19:         if (!res.ok) throw new Error('Error fetching data');
20:
21:         const data: Measurement[] = await res.json();
22:         setMeasurements(data);
23:       } catch (err) {
24:         setError((err as Error).message);
25:       }
26:     }
27:     fetchData();
28:     if (error) return <div>Error: {error}</div>;
29:     function getHealthyAndCurrent(measurements: Measurement[])
30:     {
```

```
27:     const grouped: Record<string, { healthy?: Measurement;
    current?: Measurement }> = {};
28:     for (const m of measurements) {
29:         const coord = m.coordinate;
30:         if (!grouped[coord]) grouped[coord] = { healthy: m, current:
    m };
31:         else {
32:             if (new Date(m.startMeasurement).getTime() < new Date(grouped[
    coord].healthy.startMeasurement).getTime()) {
33:                 if (new Date(m.endMeasurement).getTime() > new Date(grouped[
    coord].current.endMeasurement).getTime()) {
34:                     grouped[coord].healthy = m;
35:                     grouped[coord].current = m;
36:                 }
37:             }
38:         }
39:     }
40:     const grouped = getHealthyAndCurrent(measurements);
41:     const combineData = (healthy?: Measurement, current?: Measurement)
    => {
42:         if (!healthy and !current) return [];
43:         const length = healthy ? healthy.values.length : current!.values
    .length;
44:         const combined: { frequency: number; healthyValue?: number;
    currentValue?: number }[] = [];
45:         for (let i = 0; i < length; i++) {
46:             combined.push({
47:                 frequency: healthy?.frequencies[i] ?? current!.frequencies[i],
48:                 healthyValue: healthy?.values[i],
49:                 currentValue: current?.values[i]
50:             });
51:         }
52:     }
53: }
```

```

51:     }
52:     return combined;
53: };
54:
55: const xData = combineData(grouped.X?.healthy, grouped.X?.current);
56: const yData = combineData(grouped.Y?.healthy, grouped.Y?.current);
57: const zData = combineData(grouped.Z?.healthy, grouped.Z?.current);
58: const magnitudeData = combineData(grouped.Magnitude?.healthy,
    grouped.Magnitude?.current);
59:
60: return (
61:     <div>
62:         <h1>Vibration Frequency Dashboard</h1>
63:         {measurements.length === 0 ? <p>Loading measurements...</p>
        : <> }
64:         /* X Axis */
65:         <h2>X Axis</h2>
66:         <ResponsiveContainer width="100per cent" height=700><LineCha
            data=xData><CartesianGrid strokeDasharray="3 3" /><XAxis dataKey="fr
            label={{ value: 'Frequency (Hz)', position: 'insideBottom',
            offset: -5 }} /><YAxis label={{ value: 'Amplitude', angle: -90,
            position: 'insideLeft' }} /><Tooltip /><Legend /><Line type="monoton
            dataKey="healthyValue" stroke="8884d8" name="Healthy State"
            strokeWidth=3/><Line type="monotone" dataKey="currentValue"
            stroke="82ca9d" name="Current State" /></LineChart></ResponsiveConta
67:         /* Y, Z, Magnitude axes follow the same pattern as X
            Axis */
68:     </> }

```

69: </div>);

Auxiliary Vercel Algorithm

```
1: Import supabaseAdmin from '../lib/supabase'
2:
3: Function GET():
4:   Try:
5:     Log "Fetching data from Supabase..."
6:     Query measurements table using supabaseAdmin
7:     If error occurs:
8:       Log error details
9:       Return error response with status 500
10:    Log fetched data
11:    Return JSON response with data
12:   Catch error:
13:     Log unexpected error
14:     Extract error message
15:     Return error response with status 500
16: End Function
```